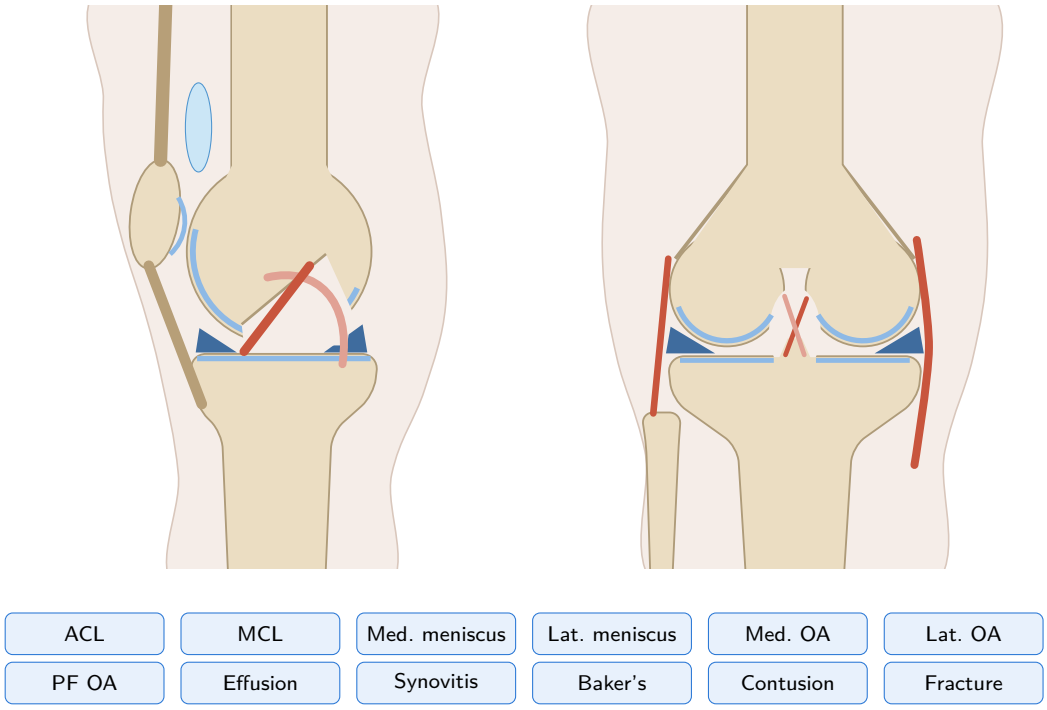
RSNA 2026 Knee Abnormality Detection

Knee MRI, Labels and Leaderboards

A beginner's script: the data, everything we tried, and how the winners did it



David Pilný

written together with Claude (Anthropic)

October 2026

Twelve findings, one knee MRI exam per row, 4,407 training exams and only 58 of them labelled by doctors. This script explains how to turn that into a model, what we built, what scored what, and what the team that won did differently.

Contents

How to read this script **vii**

I The problem **1**

1 The competition in one chapter **2**

1.1 The task in one picture . . . 2
1.2 The twelve findings . . . 3
1.3 What "code competition" means . . . 3
1.4 The score: macro ROC-AUC . . . 3
1.5 Two leaderboards . . . 4
1.6 The efficiency track . . . 4
1.7 The calendar . . . 5

2 The knee and its twelve findings **6**

2.1 The parts of a knee . . . 6
2.2 Three compartments . . . 7
2.3 The twelve findings, one card each . . . 8
2.4 "Positive" means "past a threshold" . . . 9
2.5 What it looks like on real images . . . 10

3 MRI in fifteen minutes **11**

3.1 How MRI makes a picture, in three sentences . . . 11
3.2 Slices and stacks . . . 11
3.3 Three directions . . . 12
3.4 Sequences: what is bright and what is dark . . . 12
3.5 A typical knee exam . . . 13
3.6 Voxels are not cubes . . . 14
3.7 MRI has no units . . . 14

4 The data **16**

4.1 Folders: study, series, slice . . . 16
4.2 One DICOM file . . . 16

4.3 The tables . . . 17
4.4 The numbers . . . 18
4.5 Who labelled what . . . 18
4.6 How common is each finding? . . . 19
4.7 The test set has no reports . . . 19

II Machine-learning fundamentals 21

5 The machine-learning toolbox 22
5.1 A model is a function with knobs . . . 22
5.2 How a CNN sees: filters . . . 22
5.3 Backbone and head . . . 23
5.4 2D, 2.5D or 3D? . . . 23
5.5 From many slices to one answer: pooling . . . 24
5.6 From numbers to probabilities . . . 24
5.7 The loss: how wrong is the model? . . . 25
5.8 The training loop . . . 25
5.9 Overfitting and augmentation . . . 26
5.10 Validation: testing honestly . . . 26
5.11 Ensembles and test-time augmentation . . . 27

6 Measuring success: ROC-AUC 28
6.1 Why not just count correct answers? . . . 28
6.2 AUC as a ranking game . . . 28
6.3 The ROC curve . . . 29
6.4 Real curves from our model . . . 29
6.5 Twelve AUCs become one: the macro average . . . 30
6.6 Only the order matters . . . 30
6.7 How noisy is an AUC? . . . 31

III What we built 33

7 Step 1: reports into labels 34
7.1 What a language model does . . . 34
7.2 Why an open model, inside Kaggle? . . . 35

7.3 Version 3: the first labeller . . . 35
7.4 Version 4: four changes . . . 35
7.5 Results . . . 37
7.6 What the labeller actually answered . . . 38
7.7 What reports cannot do . . . 38

8 Step 2: images into a cache **40**
8.1 Why a cache? . . . 40
8.2 a. Pick one series per role . . . 40
8.3 b. Sort the slices . . . 41
8.4 c. One orientation for everybody . . . 41
8.5 d. Left or right knee: written down, not changed . . . 41
8.6 e. Normalise the brightness . . . 42
8.7 f. Crop the knee . . . 42
8.8 g. Resize to one shape . . . 42
8.9 The same code at test time . . . 43

9 Step 3: the model **44**
9.1 The architecture . . . 44
9.2 Loss and labels . . . 45
9.3 Augmentation, on the GPU . . . 45
9.4 Training settings . . . 46
9.5 Three hard rules from Kaggle's GPUs . . . 46
9.6 First results of the 256 px model . . . 47

10 Step 4: the submission **48**
10.1 The flow . . . 48
10.2 Safety net first . . . 48
10.3 Offline installs . . . 49
10.4 Model groups and rank blending . . . 49
10.5 Plugging into a public notebook: the "member cell" . . . 49

11 Everything we tried **51**
11.1 The scoreboard . . . 51
11.2 Experiment by experiment . . . 52
11.3 Why our models could not lift the public ensemble . . . 54

11.4 How it ended 54

IV The winners **56**

12 The winners, part 1: labels **57**

12.1 The big picture 57
12.2 An agent, not a prompt 57
12.3 The agent's workspace 58
12.4 Evidence first, then a number 58
12.5 How good were the labels? 59
12.6 Judging labels by more than AUC 59
12.7 A guide that learns from the doctors 60
12.8 Small decisions that mattered 61

13 The winners, part 2: models **62**

13.1 Two kinds of boxes 62
13.2 From a few agent boxes to boxes everywhere 62
13.3 Box pooling 63
13.4 Learning from boxes and reports during training only 64
13.5 The input: more of everything 64
13.6 What each piece was worth 65
13.7 Training 65
13.8 A ruler they could trust 65
13.9 The final ensemble 66
13.10 The efficiency notebook: 0.954 in four minutes 66
13.11 How they worked 67

14 Us vs. them **68**

14.1 Side by side 68
14.2 Where the gap came from 69
14.3 What we got right 69
14.4 Seven lessons 70
14.5 If we started again 70

A Glossary **72**

B Answers to the checkpoints **74**

C Image credits and sources **77**

How to read this script

This script is for someone who knows how to program but is new to medical imaging, to Kaggle competitions, or to both. It follows one competition from start to finish: what the data looks like, which ideas we tried and why, what each one scored, and what the team that finished first did.

You do not need to read it in order. Every chapter starts with a one-minute summary and ends with a checkpoint. The pictures carry most of the story.

The boxes

The text is broken up by coloured boxes. Each box has an icon and a word, so the meaning survives a black-and-white printer.

In one minute

Three to five bullets at the start of every chapter. If you only read these, you get the whole story.

Key idea The one sentence to remember from a section.

Why? The reason behind a choice. Read these when something feels arbitrary.

Word check A new word, explained in plain language the first time it appears. All of them are collected in the glossary (Appendix A).

Watch out A mistake that is easy to make, often one we made ourselves.

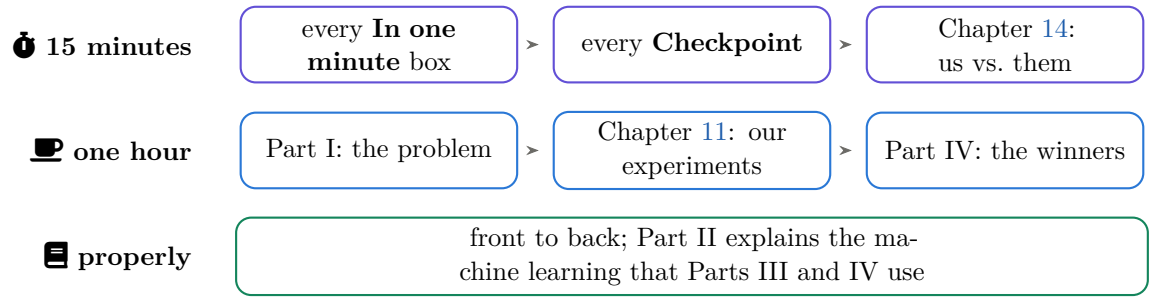
In numbers The figures that matter, in one place.

Example A small worked example, usually about one imaginary knee called **Study K**.

Checkpoint

- A short list of what you should now be able to explain.
- Two or three questions to test yourself. The answers are in Appendix B.

Three ways through



The whole story on one page

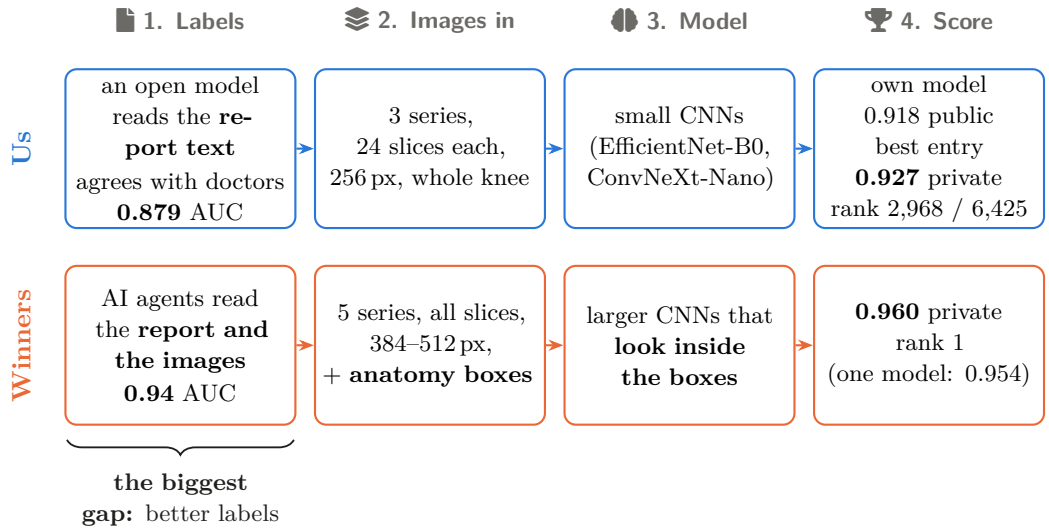


Figure 1: Our pipeline and the winners' pipeline, side by side. Both have the same four parts. The largest difference is in the first one: how good the training labels are.

The story in six sentences.

1. The task: look at one knee MRI exam and give a probability for each of 12 problems (a torn ligament, a torn meniscus, fluid in the joint, …).
2. Only 58 of the 4,407 training exams were labelled by doctors; the rest only have a free-text report.
3. So everybody first had to **turn reports into labels**, and then train an image model on those labels.
4. We did this with an open language model that read the reports, and trained small image models: 0.918 on our own, 0.927 on the final leaderboard with the help of a public notebook.
5. The winners let AI agents read the reports *and look at the images*, and also asked them to draw boxes around the anatomy; their models scored 0.960.
6. Most of the gap comes from the labels, a smaller part from telling the model where to look.

Part I

The problem

Chapter 1

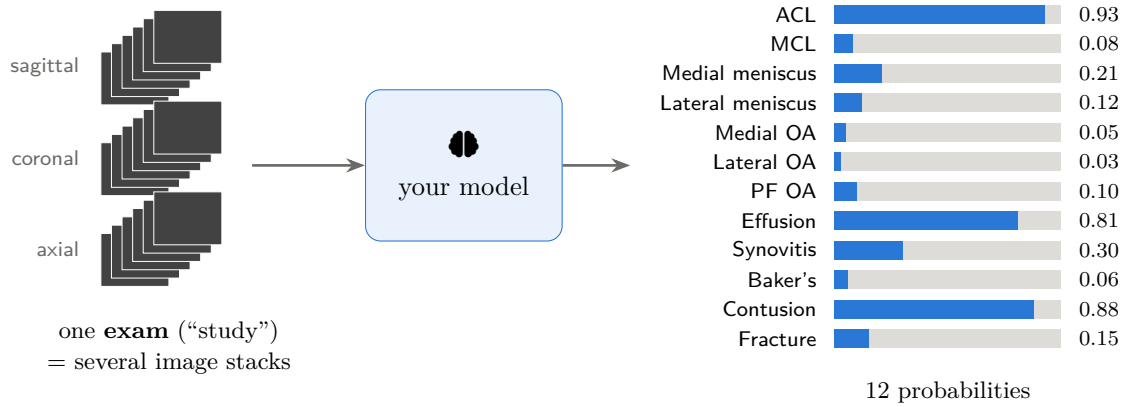
The competition in one chapter

In one minute

- **Input:** one knee MRI exam (several 3D image stacks). **Output:** 12 probabilities, one per finding.
- **Score:** ROC-AUC per finding, averaged over the 12 (rare findings count as much as common ones).
- **Code competition:** you submit a notebook; Kaggle runs it offline on ∼1,300 hidden exams, at most 9 hours.
- **Two leaderboards:** public (30 % of the test exams, visible during the competition) and private (the other 70 %, revealed at the end, the one that counts).

1.1 The task in one picture

The organiser is the RSNA (Radiological Society of North America); the competition ran on Kaggle in autumn 2026 with $77,000 in prizes. The job: build a program that looks at a knee MRI exam and says, for 12 possible problems, how likely each one is.



one **exam** (“study”) = several image stacks

12 probabilities

Figure 1.1: The task. The example output is what a good model might say for **Study K**, an imaginary footballer who twisted their knee: a torn ACL (0.93), fluid in the joint (0.81) and bruised bone (0.88).

Word check **Study** (or exam): everything scanned in one visit, for one knee. **Series**: one 3D image stack inside the study, taken in one direction with one setting (Chapter 3). **Finding**: one of the 12 things to detect.

1.2 The twelve findings

Chapter 2 explains each one with a drawing. For now, just the list:

Ligaments & menisci ("soft parts that tear")		Joint wear, fluid & bone	
ACL	anterior cruciate ligament torn	**Medial OA**	cartilage worn away, inner side
MCL	medial collateral ligament torn	**Lateral OA**	cartilage worn away, outer side
Medial meniscus	inner shock absorber torn	**PF OA**	cartilage worn behind the kneecap
Lateral meniscus	outer shock absorber torn	**Effusion**	too much fluid in the joint
		Synovitis	inflamed joint lining
		Baker's	fluid cyst behind the knee
		Contusion	bruised bone
		Fracture	broken bone

A study can have none, one or many of them. Study K has three. This is called a **multi-label** problem: 12 independent yes/no questions about the same images.

1.3 What "code competition" means

In many Kaggle competitions you download the test data and upload a file of answers. Here you never see the test images. You submit a **notebook** (a program), and Kaggle runs it on its own machine.

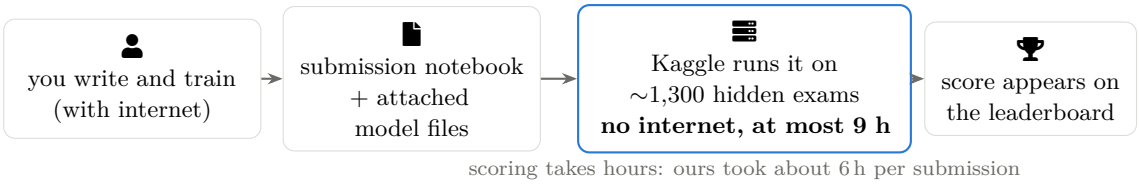


Figure 1.2: A code competition. Everything the notebook needs (model weights, extra Python packages) must be attached in advance as Kaggle *datasets*, because the scoring run is offline.

Watch out A notebook that crashes on the hidden data gives no score, and you only find out hours later. That is why every serious submission first writes a **safety-net** answer file (a constant guess) and then improves on it study by study (Chapter 10).

1.4 The score: macro ROC-AUC

For each finding Kaggle computes the **ROC-AUC**, then averages the 12 numbers. You will meet AUC properly in Chapter 6; for now one sentence is enough:

Key idea AUC = the chance that the model gives a randomly chosen *sick* knee a higher score than a randomly chosen *healthy* knee. 0.5 is coin-flipping, 1.0 is perfect. Only the *order* of the scores matters.

Because the 12 AUCs are averaged with equal weight (a **macro** average), a rare finding like MCL tears counts exactly as much as a common one like effusion.

1.5 Two leaderboards

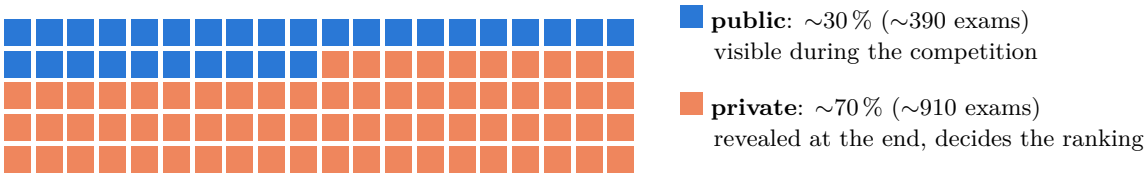


Figure 1.3: The hidden test set is split once, at random. Each square is 1 % of the ∼1,300 test exams.

Why? **Why two leaderboards?** If the score you see were the final score, you could submit many slightly different versions and keep the luckiest one. This is called **overfitting the public leaderboard**. The private part stops it: a choice that was only lucky on 390 exams does not stay lucky on the other 910.

In numbers With ∼390 public exams the public score wobbles by about ±0.005 from luck alone. Differences smaller than that are noise. At the end you **select two submissions**; the better of the two on the private leaderboard is your final result.

1.6 The efficiency track

A second, smaller prize ($7k / $6k / $5k) rewarded fast *and* good submissions:

$$\text{Efficiency} = \frac{\text{AUC}}{\text{Benchmark} - \text{maxAUC}} + \frac{\text{runtime in seconds}}{32{,}400} \qquad \text{(lower is better)}$$

The benchmark is below the best AUC, so the first term is negative: a higher AUC makes it more negative. In practice this works out to roughly $-2.16 \cdot \text{AUC} + \text{hours}/9$, so **one extra hour of runtime costs about as much as 0.05 AUC**. A 4-minute notebook at 0.954 (the winners' single model) beats a 6-hour notebook at 0.96 easily.

1.7 The calendar

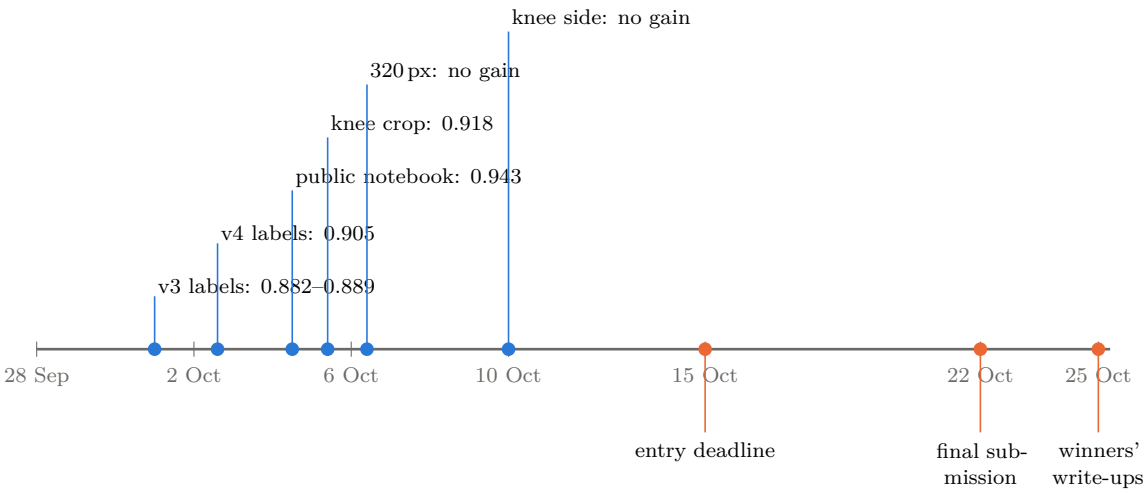


Figure 1.4: Our project in time (blue, public leaderboard scores) and the competition deadlines (orange). Our active phase was the first two weeks of October.

Checkpoint

- ☑ I can say what goes in (one exam) and what comes out (12 probabilities).
- ☑ I know why the public leaderboard is not the final score.
- ? Q1.1: Why does a crashing notebook hurt more in a code competition than in a normal one?
- ? Q1.2: The public score of your new version is 0.003 higher. Is it better?
- ? Q1.3: In the efficiency track, is it worth adding a model that adds 0.01 AUC but one hour of runtime?

Chapter 2

The knee and its twelve findings

In one minute

- The knee is where the thigh bone (femur) meets the shin bone (tibia); the kneecap (patella) glides in front.
- Four kinds of parts can go wrong: **ropes** (ligaments), **cushions** (menisci), **surfaces** (cartilage), and **fluid and bone**.
- The 12 findings are those parts, split into **medial** (inner) and **lateral** (outer) where it matters.
- "Positive" means **severe enough**: a small effusion or a mild sprain counts as *negative*.

2.1 The parts of a knee

Word check **Medial** = towards the middle of the body (the inner side of the knee). **Lateral** = away from the middle (the outer side). **Anterior** = front, **posterior** = back. These words name the findings: *medial* meniscus, *lateral* OA, *anterior* cruciate ligament.

The parts in one line each.

- **Bones:** femur on top, tibia below, patella in front, fibula on the outer side of the tibia.
- **Cartilage:** a smooth layer (2–4 mm) on the bone ends, so they glide.
- **Menisci:** two C-shaped rubbery cushions between femur and tibia, one medial, one lateral.
- **Ligaments:** ropes that hold the bones together. The two **cruciates** (ACL, PCL) cross inside the knee; the two **collaterals** (MCL inner, LCL outer) run along the sides.
- **Synovium and joint fluid:** the joint is a sealed bag lined by the synovium, with a little lubricating fluid inside. The bag has a big pocket above the kneecap where extra fluid collects.

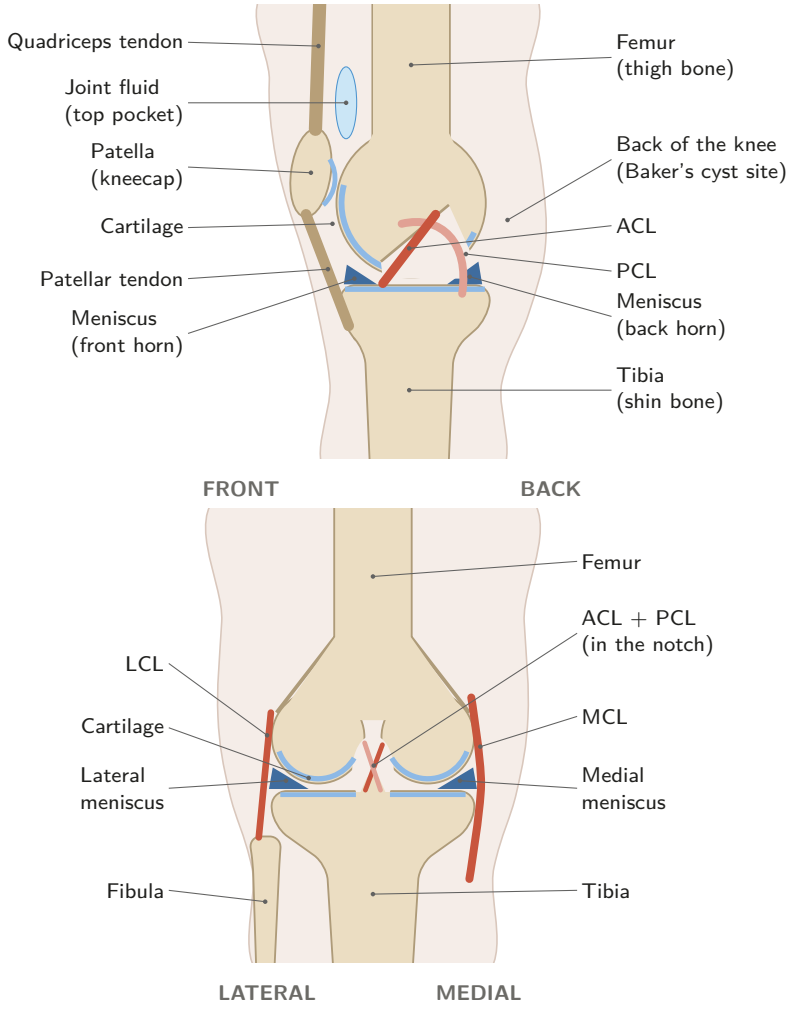


Figure 2.1: Left: a knee seen from the side (front of the leg on the left, as in the MRI images later). Right: a right knee seen from the front. Blue wedges = menisci, light blue = cartilage, red = ligaments.

2.2 Three compartments

Doctors split the knee into three areas where two cartilage surfaces touch. Three of our findings are "osteoarthritis in compartment X".

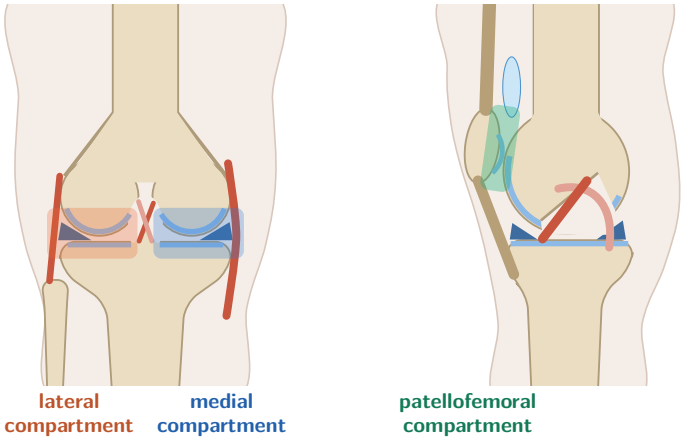


Figure 2.2: Medial OA and **Lateral OA** are about the two tibiofemoral compartments (femur on tibia, left). **PF OA** is about the patellofemoral compartment, where the kneecap glides on the femur (right).

2.3 The twelve findings, one card each

Every card says what the finding is, when it counts as **positive** in this competition, and where it is easiest to see (the planes are explained in Chapter 3).

ACL (anterior cruciate ligament)

The rope in the middle of the knee that stops the shin bone sliding forward. Torn by twisting, typical in football and skiing.

Positive if: high-grade partial or complete tear (more than half the fibres).
Negative: sprain, low-grade partial tear, signal change only.
Best seen: sagittal images.

MCL (medial collateral ligament)

The rope on the inner side. Torn when the knee is hit from the outside.

Positive if: acute high-grade or complete tear.
Negative: low-grade sprain, old scarring.
Best seen: coronal images.

Medial meniscus

The inner cushion. Tears from twisting, or from wear in older knees.

Positive if: a tear reaching the meniscus surface (seen on $\geq$2 images), or a deformed meniscus.
Negative: degeneration inside the meniscus that does not reach the surface.
Best seen: sagittal and coronal.

Lateral meniscus

The outer cushion. Same as the medial one, on the other side.

Positive if: same rule as the medial meniscus.
Negative: same rule.
Best seen: sagittal and coronal.

Medial OA (osteoarthritis)

Cartilage worn away between femur and tibia on the inner side. The most common place for knee arthritis.

Positive if: an area of about 1 cm or more where more than half of the cartilage thickness is lost.
Negative: softening, thin fissures, small defects.
Best seen: coronal and sagittal.

Lateral OA

The same wear on the outer side.

Positive if: same rule, lateral compartment.
Negative: same rule.
Best seen: coronal and sagittal.

PF OA (patellofemoral)

Cartilage worn between the kneecap and the groove of the femur.

Positive if: same rule, patellofemoral compartment.
Negative: same rule.
Best seen: axial and sagittal.

Effusion

Too much fluid in the joint bag. A sign that something inside is irritated or injured.

Positive if: moderate or large amount.
Negative: trace, small or “physiological” fluid.
Best seen: sagittal (pocket above the kneecap) and axial.

Synovitis

The lining of the joint is thickened and inflamed. Hard to see without contrast dye; reports often do not mention it.

Positive if: thickened, inflamed synovium.
Negative: normal lining; fluid alone is not synovitis.
Best seen: axial and sagittal.

Baker’s cyst

A fluid sac behind the knee, a bulge of the joint bag between two muscles.

Positive if: moderate or large cyst.
Negative: small or tiny cyst.
Best seen: axial and sagittal.

Contusion (bone bruise)

Bleeding and swelling inside the bone after an impact. Bright on fat-suppressed images. Classic companion of an ACL tear.

Positive if: impact bone-marrow edema without a fracture line.
Negative: edema from wear or overload.
Best seen: fat-suppressed fluid-sensitive images, any plane.

Fracture

A broken bone.

Positive if: an acute fracture line or break in the outer bone layer (cortex).
Negative: old healed fractures.
Best seen: a line on T1 or PD images, with edema around it on fat-suppressed images.

2.4 “Positive” means “past a threshold”

The doctors who made the reference labels used strict rules: a finding only counts if it is severe enough, and a borderline case counts as **negative**. This matters a lot, because a radiology report mentions small things too (“small effusion”, “grade 1 sprain”).

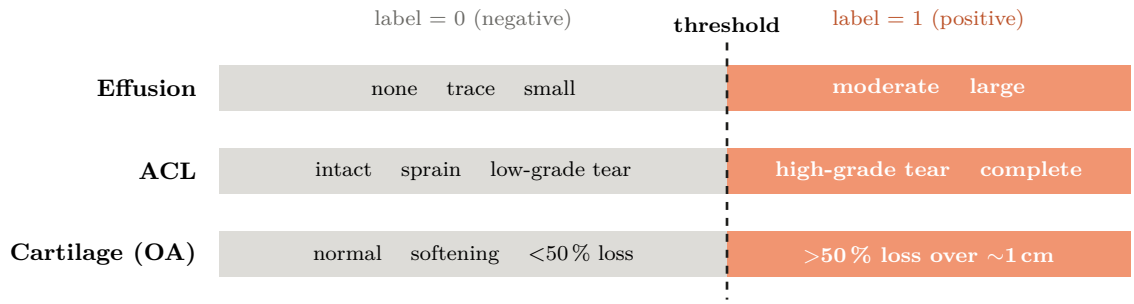


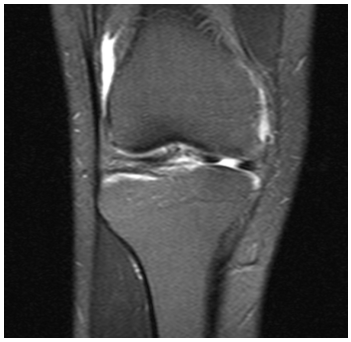
Figure 2.3: Severity scales. Everything left of the dashed line is a *negative* in this competition, even though a report may describe it as a finding. Borderline cases also go to the left.

Watch out Our first labeller (Chapter 7) used the opposite rule, “any grade counts”. That one sentence cost us about 0.07 in label quality. **Read the organisers’ definitions before you label anything.**

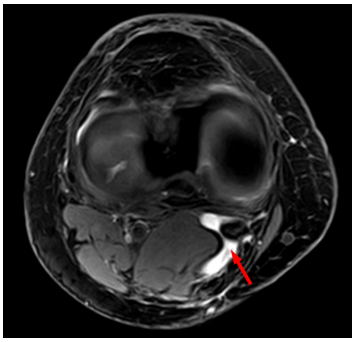
2.5 What it looks like on real images



(a) ACL rupture, sagittal. A normal ACL is a dark, straight band; a ruptured one is interrupted or missing. Bright fluid fills the joint.



(b) Meniscus tear, coronal. A normal meniscus is a dark wedge; a tear shows up as a bright line inside it.



(c) Baker's cyst, axial (a slice across the leg). The arrow marks the channel between the cyst and the joint.

Figure 2.4: Three findings on real MRI images. All three images are fat-suppressed, so fluid is bright and fat is dark (Chapter 3). Images: Hellerhoff and Ciernik M, Wikimedia Commons, CC BY-SA (Appendix C).

Checkpoint

- ☑ I can point to the femur, tibia, patella, both menisci, the ACL and the MCL on a drawing.
- ☑ I know what medial, lateral and "compartment" mean.
- ? Q2.1: A report says "small joint effusion". Is Effusion positive?
- ? Q2.2: Which three findings share one rule about cartilage, and how do they differ?
- ? Q2.3: Why might a model confuse Medial and Lateral Meniscus if it cannot tell a left knee from a right knee?

Chapter 3

MRI in fifteen minutes

In one minute

- An MRI **series** is a stack of 2D slices; a knee **exam** has several series.
- Each series is cut in one of three directions: **sagittal** (side view), **coronal** (front view), **axial** (cross-section).
- The scanner settings (the **sequence**) decide what is bright. **Fluid-sensitive, fat-suppressed** series make fluid, swelling and tears bright: they are the ones that show most findings.
- Pixels are small (∼0.4 mm) but slices are thick (∼3.5 mm), and brightness has **no units**.

3.1 How MRI makes a picture, in three sentences

The scanner is a strong magnet. Radio pulses make the hydrogen in the body's water and fat send back a weak signal, and the scanner measures it point by point. How bright a tissue looks depends on how much water and fat it holds *and* on the timing of the radio pulses, so the same knee can be imaged in very different ways.

3.2 Slices and stacks

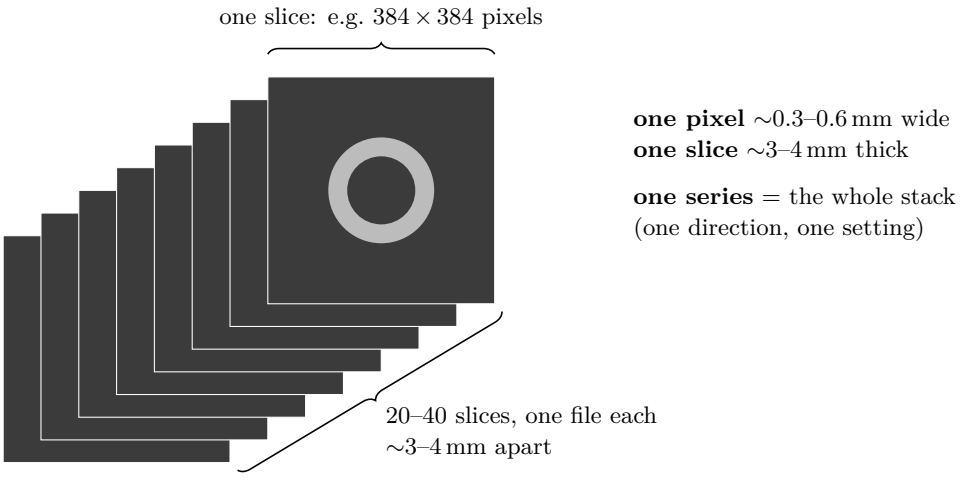
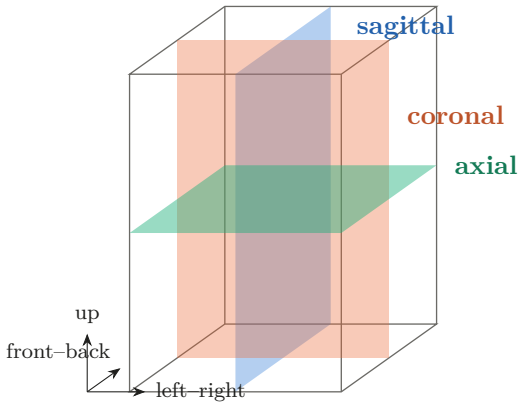


Figure 3.1: A series is a stack of slices. In the competition files every slice is a separate file.

3.3 Three directions

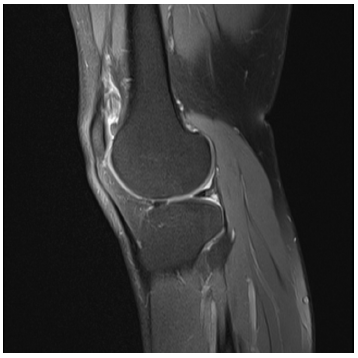


Sagittal: slices from left to right. Each slice is a side view. Best for the ACL, the front and back of the menisci, effusion.

Coronal: slices from front to back. Each slice is a front view. Best for the MCL, the middle of the menisci, medial vs. lateral cartilage.

Axial: slices from bottom to top. Each slice is a cross-section of the leg. Best for the kneecap (PF OA), Baker's cyst, synovitis.

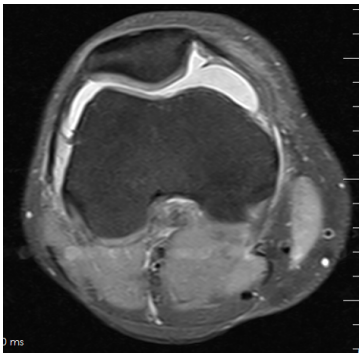
Figure 3.2: The three directions. Each one shows full detail within its slices and coarse detail across them, so an exam needs all three.



(a) Sagittal (front of the knee on the left).



(b) Coronal.

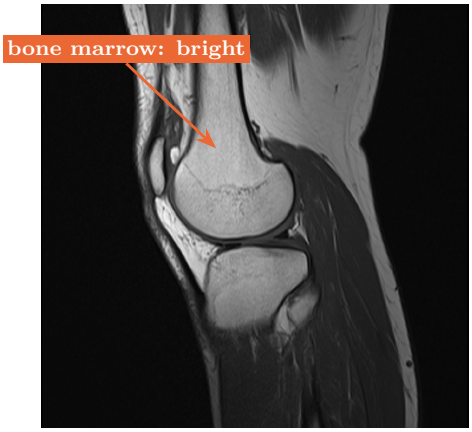


(c) Axial (kneecap at the top).

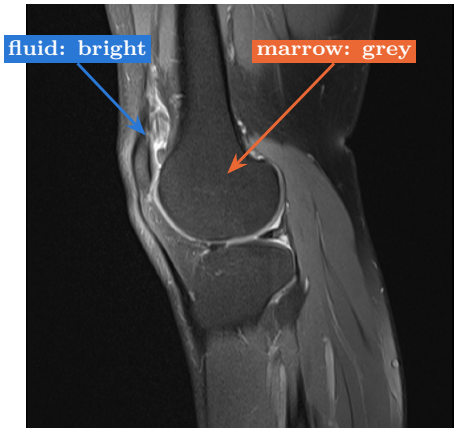
Figure 3.3: One slice from each direction (three different people). Compare them with the drawings in Figure 2.1. Images: Ptrump16 and Nevit Dilmen, Wikimedia Commons, CC BY-SA.

3.4 Sequences: what is bright and what is dark

The radio-pulse timing is called the **sequence**. Knee exams use two families:



(a) T1: fat is bright. The bone marrow (fatty) looks white; fluid is dark. Good for anatomy and fracture lines.



(b) PD fat-suppressed: fat is switched off. Marrow turns grey; fluid and swelling stand out white. Good for tears, effusion, bone bruises.

Figure 3.4: The same knee, the same slice position, two sequences. Images: Ptrump16, Wikimedia Commons, CC BY-SA 4.0.

Tissue	T1	fluid-sensitive, fat-suppressed
fat (under the skin, bone marrow)	**bright**	dark
fluid (effusion, cysts)	dark	**bright**
swelling / edema (bone bruise)	greyish	**bright**
ligaments, tendons, menisci	dark	dark (a tear = a bright line)
cartilage, muscle	grey	grey

Key idea Most of the 12 findings are "water where it should not be": fluid in the joint, swelling in the bone, fluid seeping into a torn ligament or meniscus. A **fluid-sensitive, fat-suppressed** series makes exactly that bright. That is why our pipeline used those series first.

Word check **PD** (proton density), **T1**, **T2**: names of sequences. **Fat suppression** ("fat-sat", "FS"): a trick that removes the fat signal. **Fluid-sensitive**: a sequence where water is bright (PD or T2, usually with fat suppression). The competition's `train_series.csv` tells you, for every series, its plane, whether it is fluid-sensitive and whether fat is suppressed.

3.5 A typical knee exam

Series	Plane	Sequence	Used by us?
sag fluid	sagittal	PD fat-suppressed	✓ role `sag_fs`
cor fluid	coronal	PD or T2 fat-suppressed	✓ role `cor_fs`
ax fluid	axial	PD fat-suppressed	✓ role `ax_fs`
sag structural	sagittal	T1 or PD without fat suppression	✗ (the winners did)
cor structural	coronal	T1	✗ (the winners did)

The training set has 24,371 series for 4,407 exams: about **5.5 series per exam**. Sites differ: some scan more, some fewer, and they name and store them differently.

3.6 Voxels are not cubes

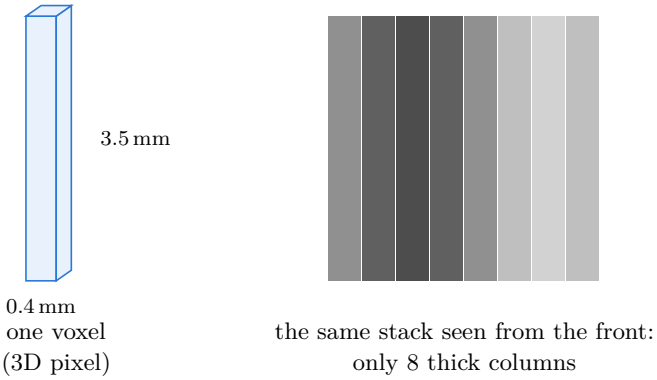


Figure 3.5: Within a slice the detail is fine; across slices it is about ten times coarser. Turning a sagittal stack into a front view gives a blocky picture.

Why? This is why most solutions (ours and the winners') use **2D image models on single slices or on a few neighbouring slices** ("2.5D", Chapter 5) instead of true 3D models, and why the exam includes three directions: each direction is sharp where the others are coarse.

3.7 MRI has no units

In a CT scan, water is always 0 on the brightness scale. In MRI the raw numbers depend on the scanner, the coil, the sequence and the patient. Only *relative* brightness inside one series means anything.

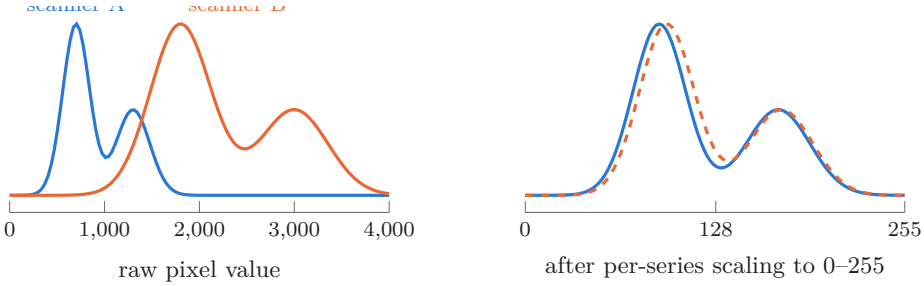


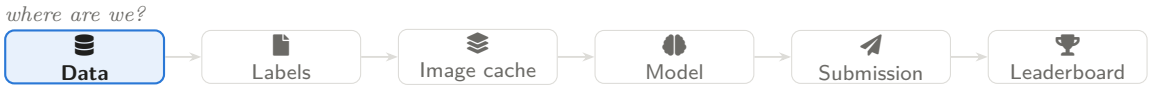
Figure 3.6: The same two tissues on two scanners (schematic). Raw values are not comparable (left). Scaling every series by its own percentiles lines them up (right). Chapter 8 shows how we did it.

Checkpoint

- ☑ I can name the three directions and say what each one is good for.
- ☑ I know why fluid-sensitive, fat-suppressed series matter for this task.
- ? Q3.1: On a T1 image, is a bone bruise easy to see? On a fat-suppressed PD image?
- ? Q3.2: Why can't you use one fixed brightness rule ("above 1,000 = fluid") for all exams?
- ? Q3.3: A model sees only sagittal images. Which finding might it struggle with most, and why?

Chapter 4

The data



In one minute

- 4,407 training exams, 24,371 series, one **DICOM** file per slice. About 1,300 hidden test exams.
- Only **58** training exams have the 12 labels from doctors ("gold"). The other **4,349** have a free-text **radiology report** in one of 12 languages, and no labels.
- The test exams have **images only**: no report.
- So the real first task is: **turn 4,349 reports into labels**, then train an image model on them.

4.1 Folders: study, series, slice

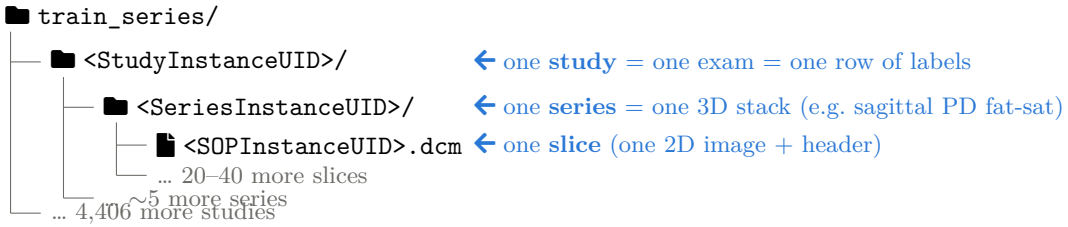


Figure 4.1: The folder tree. The long numbers (UIDs) are random identifiers; they tell you nothing by themselves. The CSV files link them to labels and to series descriptions.

4.2 One DICOM file

DICOM is the file format of medical imaging. A `.dcm` file has two parts:

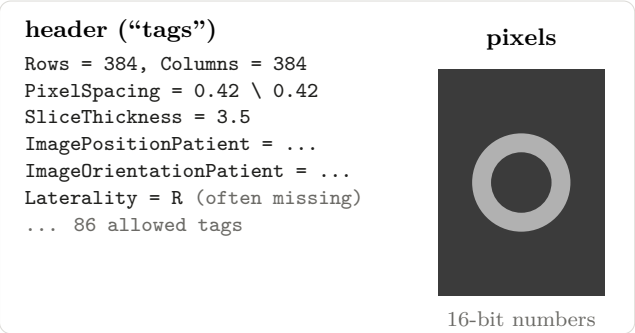


Figure 4.2: A DICOM file = a dictionary of metadata + the image. The header matters as much as the pixels: it says where the slice sits in the body, in millimetres.

Tag	Why we need it
`ImagePositionPatient`	3D position of the slice's corner (mm). Sort slices by this, never by file name.
`ImageOrientationPatient`	directions of the image rows and columns in the body. Tells the plane, and whether the image is flipped or rotated.
`PixelSpacing`, `SliceThickness`	size of a pixel and of a slice in mm. Needed to resize every series to the same physical scale.
`Laterality`	left or right knee. Missing in about half of the exams.

Watch out The competition's files keep only 86 “allowed” header tags, and sites fill them inconsistently. Every tag can be missing; read them defensively.

4.3 The tables

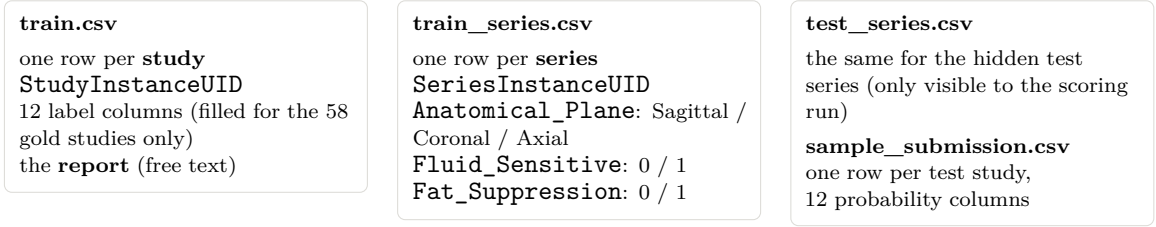


Figure 4.3: The CSV files. `train_series.csv` is precious: it already says which series is which, so we did not have to guess from the images.

4.4 The numbers

4,407	**24,371**	**~1,300**	**12**
training exams	series (~5.5 per exam)	hidden test exams	findings to predict
58	**4,349**	**12**	**977**
exams with **doctor labels**	exams with a **report** only	report languages	characters: median report length

4.5 Who labelled what

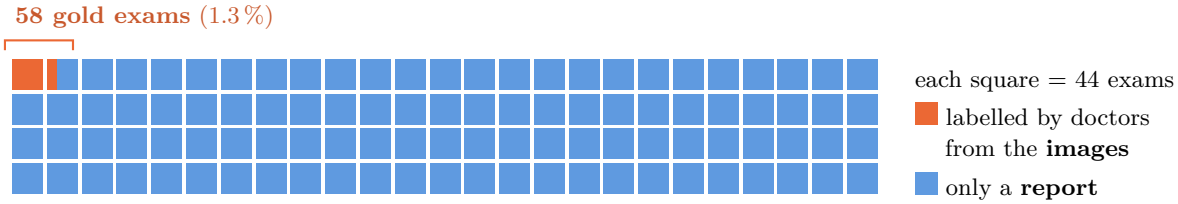


Figure 4.4: The training set. Almost everything has to be learned from reports.

The 58 gold exams. Two musculoskeletal radiologists read the *images* independently, a third settled disagreements, and they applied the strict thresholds of Figure 2.3. These are the only labels we can fully trust.

The 4,349 report-only exams. The report is what the hospital's radiologist wrote at the time, in their own language and style. It mentions what that radiologist thought worth mentioning, with their own severity words.

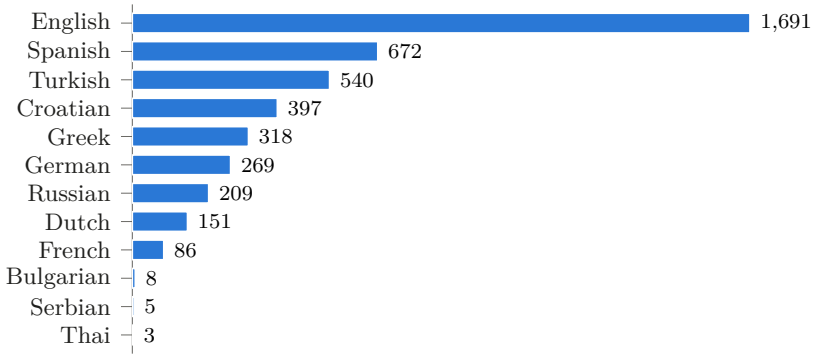


Figure 4.5: Report languages of the 4,349 report-only exams. Any labelling method has to read all of them.

Example A made-up report for our imaginary **Study K** (real reports are longer, about 1,000 characters):

> *Complete rupture of the anterior cruciate ligament. Bone contusion of the lateral femoral condyle and the posterolateral tibial plateau. Moderate joint effusion. Menisci intact. Collateral ligaments intact. Cartilage preserved.*

A careful reader would turn this into: ACL **yes**, Contusion **yes**, Effusion **yes** (moderate), both menisci **no**, MCL **no**, all three OA **no**, and … Synovitis, Baker's, Fracture: **not mentioned**. What do you do with "not mentioned"? Chapter 7 answers that.

4.6 How common is each finding?

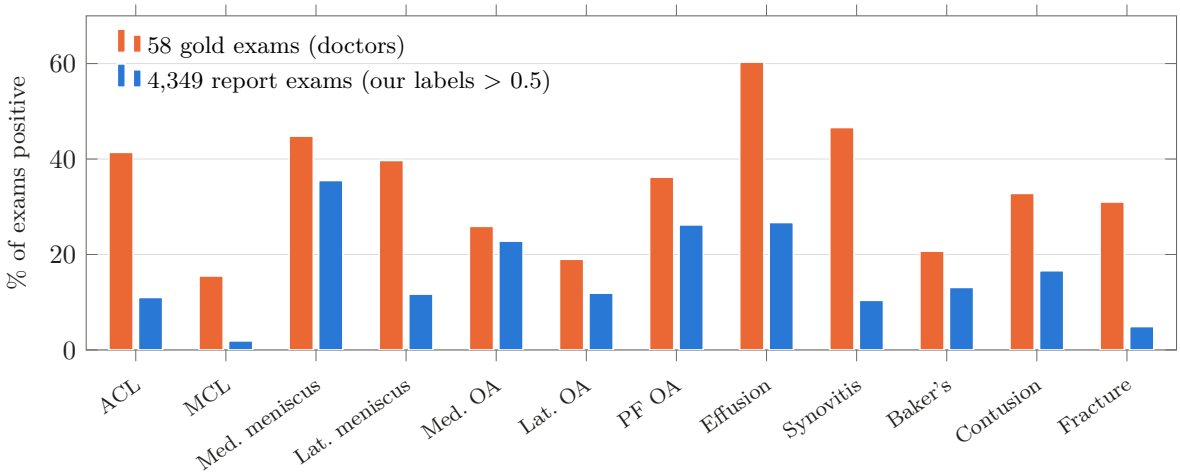


Figure 4.6: Each finding is about three times more common in the gold set than in the report set. The organisers chose the gold exams to contain enough positives. Synovitis stands out: 47 % in gold, 10 % in our labels, because reports rarely mention it.

Watch out Because the gold set is enriched, a model's *average* probabilities on gold look "too low" even when the model is fine. AUC does not care (it only looks at order), but anything that relies on calibrated probabilities would.

4.7 The test set has no reports

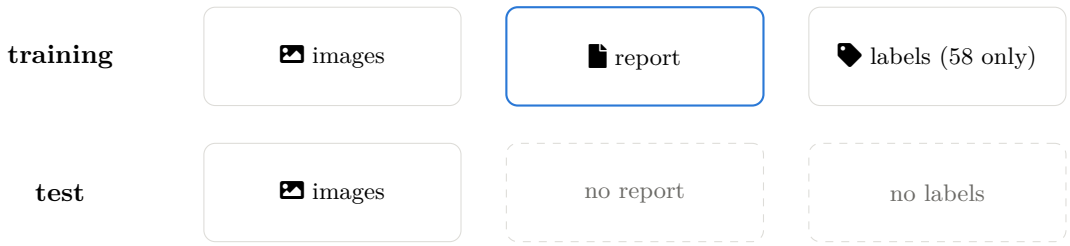


Figure 4.7: At test time only the images exist. The report can teach the model during training, but the model has to work from images alone.

Key idea Every solution in this competition has the same two steps: **(1)** turn the reports (and maybe the images) of the 4,349 exams into labels; **(2)** train an image model on those labels. The image model can never be much better than its labels. Chapter 7 is step 1, Chapters 8–9 are step 2.

Word check **Weak supervision**: training on labels that are cheaper but noisier than the real thing (here: labels read from reports instead of doctors' image labels). **Gold labels**: the trusted reference labels (here: the 58).

Checkpoint

- ☑ I can describe the path from a folder of `.dcm` files to one study with its series.
- ☑ I know which exams have labels, which have reports, and what the test set has.
- ? Q4.1: Why sort slices by `ImagePositionPatient` rather than by file name?
- ? Q4.2: Our labels say 11 % of exams have an ACL tear, the gold set says 41 %. Does that mean our labels are wrong?
- ? Q4.3: Why can't the model simply read the report at test time?

Part II

Machine-learning fundamentals

Chapter 5

The machine-learning toolbox

In one minute

- A model is a function with millions of adjustable numbers (**weights**). Training nudges them until the outputs match the labels.
- Image models are **CNNs**: a pretrained **backbone** turns an image into a feature vector, a small **head** turns features into the 12 outputs.
- Knee MRI is fed slice by slice (**2D**) or three neighbouring slices at a time (**2.5D**); the slice results are **pooled** into one answer per study.
- Honest testing needs data the model never trained on: **cross-validation**. Averaging several models (**ensembling**) almost always helps a little.

5.1 A model is a function with knobs

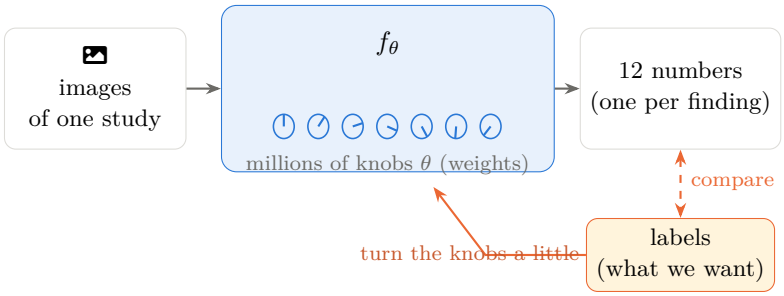


Figure 5.1: Training in one picture: compare the outputs with the labels, then turn every knob a little in the direction that reduces the difference. Repeat many thousands of times.

Word check **Weights** (parameters): the adjustable numbers inside the model. EfficientNet-B0, our main model, has about 5 million; the winners' models have 30–120 million. **Training**: adjusting them. **Inference**: using the trained model on new data.

5.2 How a CNN sees: filters

A **convolutional neural network** (CNN) looks at an image through many small **filters**. A filter is a 3×3 grid of weights that slides over the image; at each position it multiplies and adds, producing one number. The result is a new image, a **feature map**, which is bright where the filter's pattern occurs.

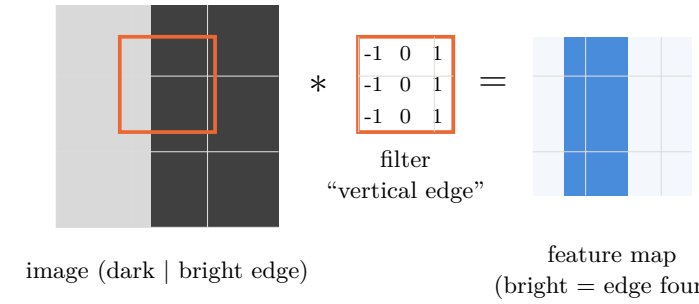


Figure 5.2: One filter finds vertical edges. A CNN learns hundreds of filters per layer and stacks dozens of layers: early layers find edges, middle layers textures, deep layers whole structures (“a meniscus”, “fluid”).

5.3 Backbone and head

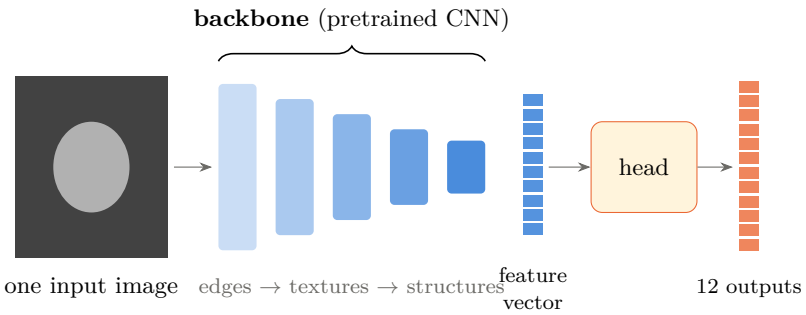


Figure 5.3: Every image model in this script has this shape. The backbone is big and pretrained; the head is small and new. “Feature vector” = a list of numbers that summarises what the backbone saw (1,280 numbers for EfficientNet-B0).

Pretrained backbones

Training a backbone from scratch needs millions of images. Instead everyone starts from a backbone trained on **ImageNet** (1.2 million photos of everyday things in 1,000 classes). Its early filters (edges, textures) are useful for any image, including MRI. We then **fine-tune**: keep training all weights on our task with a small learning rate. This is called **transfer learning**.

Backbone	Weights	Used by	Speed on a T4 GPU (ours, 256 px)
EfficientNet-B0	5 M	us	0.5 s per batch of 8 studies
ConvNeXt-Nano	15 M	us	about 1.4× slower
ConvNeXt-T(iny)	28 M	winners	2.5× slower, needed tricks to fit
MaxViT-V2-B, CoAtNet-2	100 M+	winners, public	far too slow for us on T4s

5.4 2D, 2.5D or 3D?

A knee series is a 3D stack, but the best pretrained backbones are 2D and expect 3 colour channels (red, green, blue). The common trick:

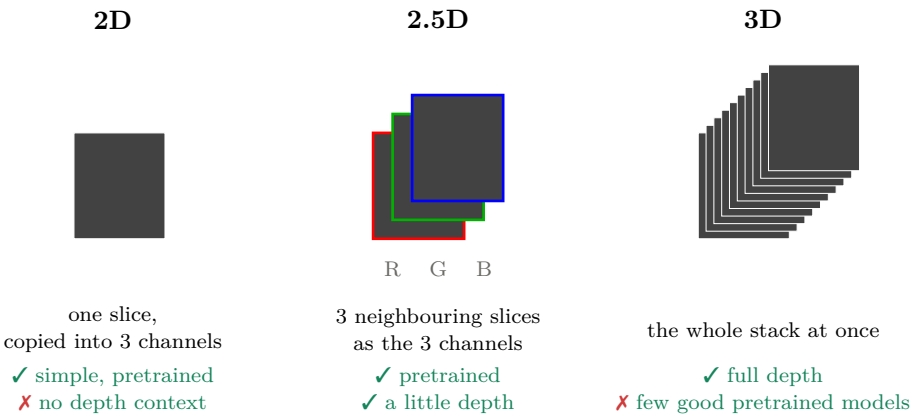
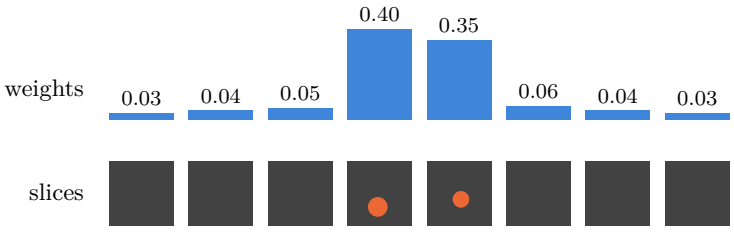


Figure 5.4: Three ways to feed slices. We used 2.5D (8 slice triplets per series). The winners found that once the model knows where to look, plain 2D works as well (Chapter 13).

5.5 From many slices to one answer: pooling

A study gives many images (we used 8 triplets × 3 series = 24). Each goes through the backbone and becomes a feature vector. They must be combined into *one* prediction per study: this is **pooling**.



attention pooling: a tiny network gives each slice a weight (they sum to 1); the study feature is the weighted average.

Here the two slices that show the tear (orange dots) get 75 % of the weight.

Figure 5.5: A tear is visible on two or three slices out of 24. A plain average would dilute it; attention pooling lets the model focus on the slices that matter. Because only the study has a label, not each slice, this setting is called **multiple-instance learning**.

5.6 From numbers to probabilities

The head outputs 12 raw numbers called **logits** (any value from $-\infty$ to $+\infty$). Each one is squeezed into a probability by the **sigmoid** function $\sigma(z) = 1/(1 + e^{-z})$:

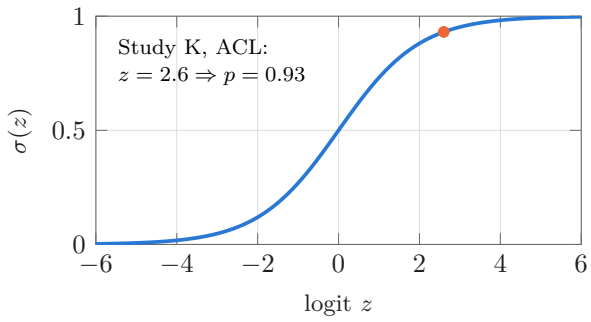


Figure 5.6: The sigmoid. Each of the 12 findings has its own sigmoid, so all 12 can be high at the same time (a *multi-label* model).

5.7 The loss: how wrong is the model?

Training needs one number that says how bad the current predictions are: the **loss**. For yes/no labels the standard is **binary cross-entropy** (BCE):

$$\mathrm{BCE}(p, y) = -\big[\, y \log p + (1 - y) \log(1 - p) \,\big]$$

where p is the predicted probability and y the label.

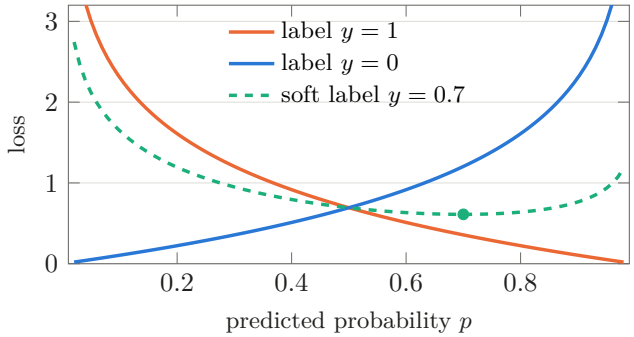


Figure 5.7: BCE punishes confident mistakes hard: predicting 0.02 for a true positive costs 3.9. With a **soft label** such as 0.7 ("probably yes"), the loss is smallest exactly at $p = 0.7$ (dot). That is how uncertain labels from reports can be used directly.

Key idea Soft labels let the model learn "probably yes" instead of a forced yes or no. Both our pipeline and the winners' trained on soft labels between 0 and 1.

Word check **Masked loss**: when a label is unknown (missing), it is simply left out of the loss, so it neither rewards nor punishes the model. Our training code did this for every missing label.

5.8 The training loop

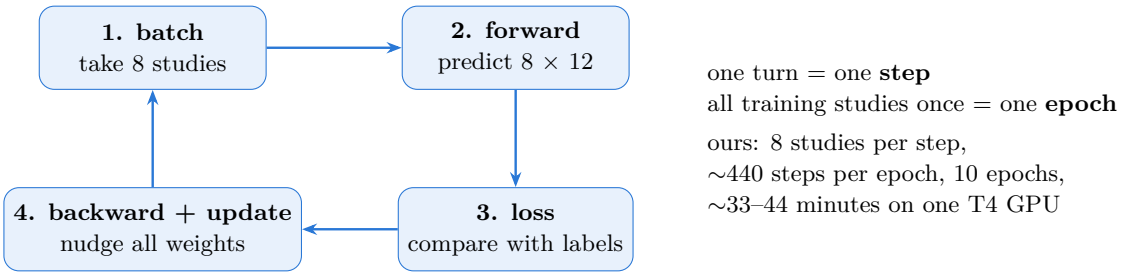


Figure 5.8: The loop. "Backward" computes, for every weight, which direction lowers the loss (the **gradient**); the **optimizer** (we and the winners used **AdamW**) takes a small step that way.

How big a step? That is the **learning rate**. It usually starts small (**warm-up**), then decreases smoothly (**cosine decay**):

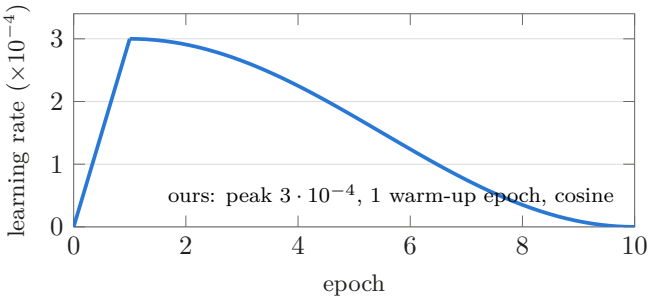


Figure 5.9: Our learning-rate schedule. The winners used a lower, constant rate for one model and a higher one with a "one-cycle" schedule for another (Chapter 13).

5.9 Overfitting and augmentation

A model with millions of knobs can simply *memorise* its training studies. It then looks perfect on them and poor on new ones: **overfitting**.

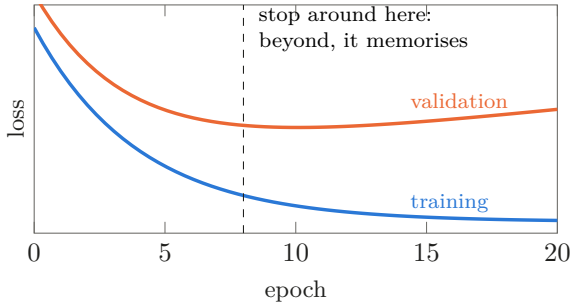


Figure 5.10: Typical curves. Training loss keeps falling; the loss on unseen studies stops improving and rises.

Augmentation fights overfitting: every time a study is shown, it is changed a little at random, so the model never sees exactly the same image twice.

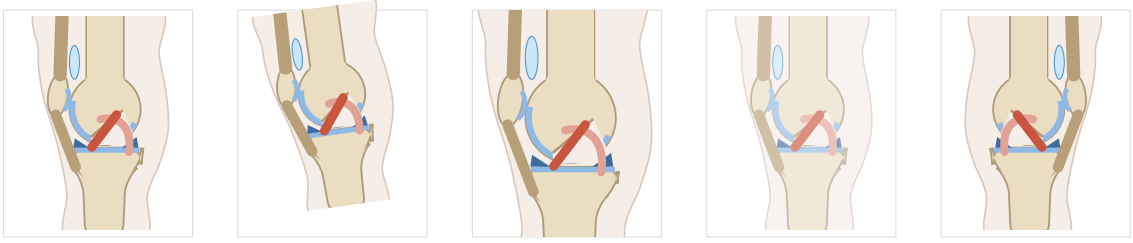


Figure 5.11: Augmentations we used: small rotation, zoom and shift, brightness/contrast/gamma changes, and mirroring the whole knee left–right. All are applied on the GPU, per series, at random.

5.10 Validation: testing honestly

To know whether a change helps, you need studies the model did not train on. **K-fold cross-validation** splits the training studies into k parts (**folds**), trains k models, each with one fold

held out:

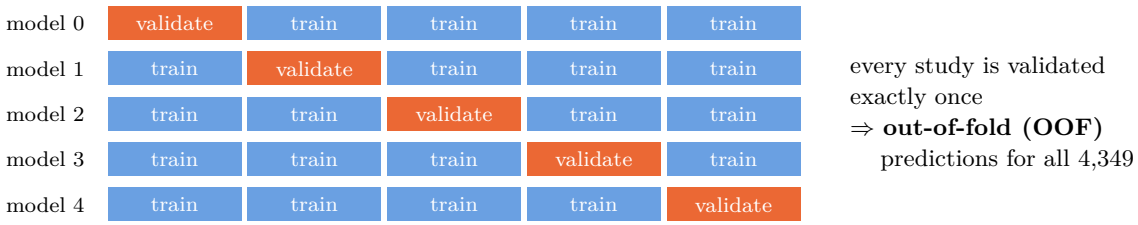


Figure 5.12: 5-fold cross-validation. At submission time the 5 models are averaged (a free ensemble).

⚠ **Watch out** If studies from the same hospital are in both the training and the validation folds, the model can recognise the hospital (its scanner, its image style) and the score looks better than it will be on new hospitals. We split **by site**: all studies of one site proxy (report language + scanner vendor, 31 groups) go into the same fold. This is called **GroupKFold**.

5.11 Ensembles and test-time augmentation

- **Ensemble**: average the predictions of several models. Their mistakes partly cancel. It helps most when the models are *different* (other labels, other inputs), and very little when they are near-copies.
- **Rank averaging**: AUC only cares about order, so instead of averaging probabilities you can convert each model's predictions to ranks (1st, 2nd, …) and average those. This stops one over-confident model from dominating.
- **Test-time augmentation** (TTA): predict on the original and on a mirrored copy, then average. We used mirror TTA.

🖩 **In numbers** Our EfficientNet-B0 and ConvNeXt-Nano models (one fold each, same images and labels) scored 0.886 and 0.898 on the 58 gold studies; their rank average scored 0.903. A small but typical ensemble gain.

Checkpoint

- ☑ I can draw backbone → feature vector → head and say which part is pretrained.
- ☑ I know what 2.5D means, what pooling does, and why soft labels work with BCE.
- ☑ I can explain k-fold and why we grouped folds by site.
- ? Q5.1: Why use 3 neighbouring slices as the 3 channels instead of one slice copied 3 times?
- ? Q5.2: A label is 0.5 ("don't know"). What prediction minimises its BCE, and what does the model learn from it?
- ? Q5.3: Two models trained on the same images with the same labels and different random seeds. Will ensembling them help a lot? Why?

Chapter 6

Measuring success: ROC-AUC

In one minute

- **Accuracy is useless** when positives are rare: "always no" scores 98 % on MCL.
- **AUC** = the share of (sick, healthy) pairs that the model puts in the right order. 0.5 = random, 1 = perfect.
- Only the **order** of the predictions matters, never their exact values.
- With few test cases AUC is **noisy**: on our 58 gold studies, ±0.02 macro is luck.

6.1 Why not just count correct answers?

Example In our report labels, only about 2 % of studies have a high-grade MCL tear. A "model" that always answers *no* is right 98 % of the time. Its accuracy is excellent, and it is completely useless: it finds no tears.

We need a score that rewards *finding the sick ones* and does not care how rare they are. That is ROC-AUC.

6.2 AUC as a ranking game

Give the model 8 studies. It returns a score for each; sort them from highest to lowest:

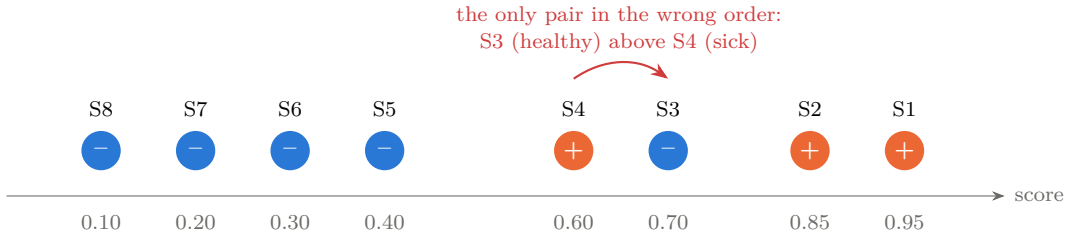


Figure 6.1: Eight studies on the model's score axis (higher score to the right). Orange + = has the finding, blue – = does not.

Now form every pair of one sick and one healthy study: $3 \times 5 = 15$ pairs. Count the pairs where the sick one has the higher score:

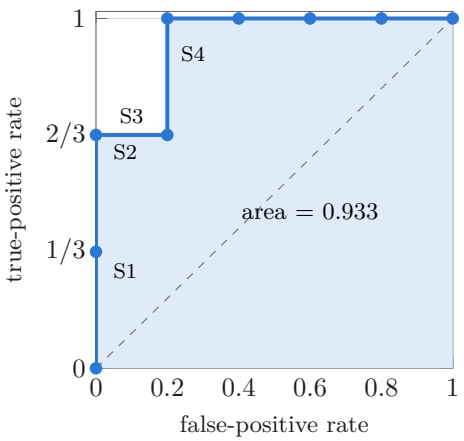
- S1 beats all 5 healthy ones: 5 S2 beats all 5: 5 S4 beats S5–S8 but not S3: 4.

$$\text{AUC} = \frac{5 + 5 + 4}{15} = \frac{14}{15} = 0.933$$

Key idea **AUC is the probability that a random sick study gets a higher score than a random healthy one.** It does not depend on how rare the finding is, and it does not depend on the scores themselves, only on their order.

6.3 The ROC curve

The same number has a picture. Move a threshold from the top score down. At each step, count the sick studies above the threshold (**true-positive rate**, TPR) and the healthy ones above it (**false-positive rate**, FPR):



Start at (0,0). Walk down the sorted list:

- a sick study: step **up** by 1/3;
- a healthy study: step **right** by 1/5.

The area under the staircase is the AUC. A perfect model goes straight up, then right (area 1). A random model follows the dashed diagonal (area 0.5).

Figure 6.2: The ROC curve of the 8-study example. ROC = "receiver operating characteristic", a name from radar engineering; AUC = "area under the curve".

6.4 Real curves from our model

These are the actual ROC curves of our v4 model (5 folds, EfficientNet-B0) on the 58 gold studies:

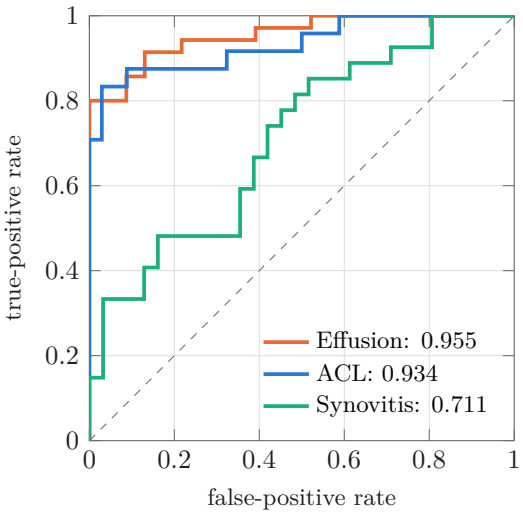


Figure 6.3: Effusion and ACL are found well; Synovitis is our weakest finding. With only 27 synovitis positives among the 58 studies the staircase has large steps: every single study moves the curve.

6.5 Twelve AUCs become one: the macro average

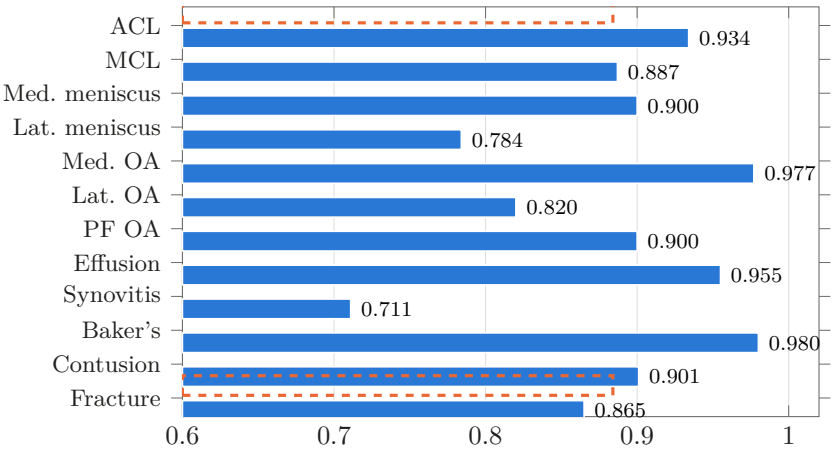


Figure 6.4: Our v4 model on the 58 gold studies, per finding. The dashed line is the macro average, 0.884. Synovitis, Lateral meniscus and Lateral OA pull it down most.

Why? The competition uses the *macro* average (each finding counts 1/12) so that a model cannot ignore the rare findings. Improving Synovitis from 0.71 to 0.81 lifts the score by $0.10/12 \approx 0.008$, as much as improving every finding by 0.008.

6.6 Only the order matters

Because AUC only looks at the order of scores:

- multiplying all predictions by 10, squaring them or passing them through any increasing function changes nothing;

- a model can be badly *calibrated* (its 0.8 does not mean "80 % chance") and still have a perfect AUC;

- to combine models, you can average their **ranks** instead of their probabilities (Chapter 10).

Watch out Calibration does not matter for the *score*, but it matters for *training labels*. A label of 0.98 that should be 0.6 teaches the model to be confidently wrong. The winners judged their labels on calibration too (Chapter 12).

6.7 How noisy is an AUC?

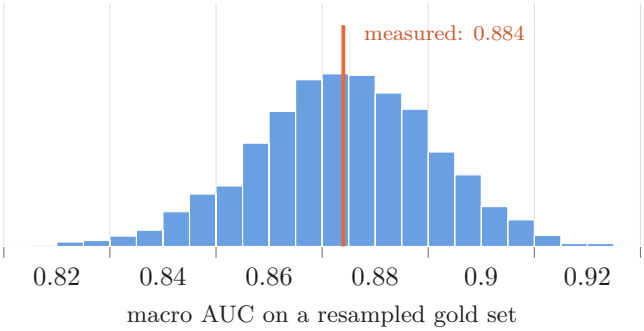


Figure 6.5: The same predictions, scored on 2,000 random resamples of the 58 gold studies (a **bootstrap**). The middle 90 % of the results spans 0.855–0.911. A difference of 0.01 between two models on gold tells you almost nothing.

In numbers

- 58 gold studies: macro AUC uncertain by about ±0.02; a single finding by ±0.05 to ±0.09 (MCL has only 9 positives).
- Public leaderboard (∼390 studies): about ±0.005.
- Out-of-fold predictions on 4,349 report-labelled studies: much more stable, but measured against *our own* labels, which have their own errors.

Key idea Every score is measured against *something*: the 58 gold labels (trustworthy but few), report labels (many but noisy) or the public leaderboard (hidden, limited submissions). Use them together and be suspicious of small differences. The winners spent real effort choosing their ruler (Chapter 13).

Checkpoint

- ☑ I can compute an AUC by counting pairs.
- ☑ I can draw a ROC curve from a sorted list.
- ☑ I know why a 0.01 gain on 58 gold studies is not evidence.
- ? Q6.1: A model outputs 0.51 for every sick study and 0.49 for every healthy one. What is its AUC?
- ? Q6.2: You add 0.3 to every prediction of one finding. What happens to the AUC?
- ? Q6.3: Which matters more for the final score: raising Baker's from 0.98 to 0.99, or Synovitis from 0.71 to 0.75?

Part III

What we built

Chapter 7

Step 1: reports into labels



In one minute

- We used an open **large language model** (LLM), running inside Kaggle, to read all 4,407 reports.
- **v3** (Qwen2.5-7B) asked for one word per finding and used "any grade counts": agreement with the doctors' gold labels **0.809** AUC.
- **v4** (Qwen2.5-32B) used the organisers' thresholds, five answer letters, and read the model's **probabilities** instead of its text: **0.879**. Leaderboard of the image model: 0.889 → 0.905.
- Reports cannot label what they do not mention: **Synovitis** is silent in 84 % of them.

7.1 What a language model does

A language model reads text split into **tokens** (word pieces) and, at every position, outputs a probability for every possible next token. Generating text means picking a token, appending it and repeating.

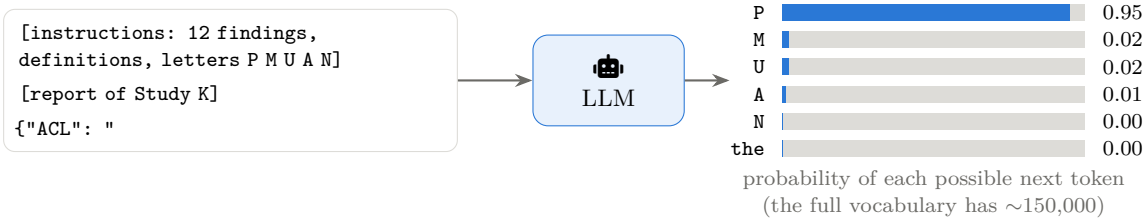


Figure 7.1: The prompt ends right where the answer for ACL should start. The model's next-token probabilities *are* the answer: 95 % "P" (present, past the threshold).

Word check **LLM**: large language model, e.g. ChatGPT, Claude, Qwen. **Open-weights model**: one whose weights you can download and run yourself (Qwen is made by Alibaba). **Prompt**: the text you give the model. **4-bit quantization**: storing each weight in 4 bits instead of 16 so a big model fits in less memory.

7.2 Why an open model, inside Kaggle?

We believed the competition's data rules forbade sending report text to hosted services such as ChatGPT or Claude, so the reports never left Kaggle. A Kaggle notebook offers two T4 GPUs with 16 GB each, for at most 12 hours per session. A 32-billion-parameter model only fits in 4-bit form (19 GB spread over both GPUs).

Watch out That belief was **wrong**. The organisers had stated in a pinned forum post that hosted LLM APIs, including for labelling reports, were allowed. We never read that post. The winners used the strongest hosted models and, crucially, let them look at the images too (Chapter 12). Lesson: **check the rules threads on day one, and treat every self-imposed rule as a question to verify.**

7.3 Version 3: the first labeller

Model	Qwen2.5-7B-Instruct (16-bit)
Prompt	"for each finding, answer `present`, `minimal`, `uncertain`, `absent` or `not_mentioned`; any grade counts"
Reading the answer	parse the generated text; map to label values 1, 0.5 or 0
Runtime	3.8 hours for 4,407 reports; 3.6 % of reports had an unparseable answer
Agreement with gold	**macro AUC 0.809**

Two problems showed up when we compared it with the 58 gold studies:

1. **Wrong definitions.** "Any grade counts" turned every small effusion and every mild sprain into a positive. The organisers' rule is the opposite (Figure 2.3).
2. **Silence became "absent".** The model almost never answered `not_mentioned` (0–2 of 58 gold reports per finding). But "the report does not mention synovitis" is not the same as "no synovitis".

7.4 Version 4: four changes

1. The organisers' thresholds, written into the prompt

For every finding the prompt lists what counts as **P** (present and past the threshold) and what counts as **M** (present but below it). Two examples:

	P (counts)	M (does not count)
Effusion	moderate or large joint effusion	small, trace, minimal, mild or physiological fluid
MCL	acute high-grade partial or complete tear (grade 2–3)	grade 1 sprain, edema around an intact ligament, old scarring

2. Five answers instead of three

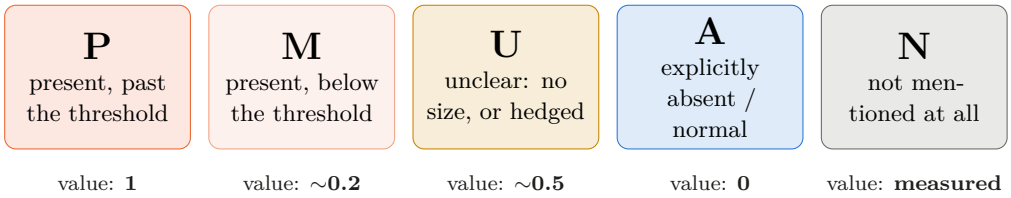


Figure 7.2: The five answer letters and their label values. The values for M, U and N were measured on the gold studies per finding (how often a gold doctor said "positive" after that answer), pulled towards the defaults shown here so that 58 studies do not overrule common sense.

Example "Not mentioned" means different things for different findings. For **Fracture** a silent report almost always means no fracture (value ∼0). For **Synovitis**, 15 of the 41 gold studies whose report was silent were positive, so N gets a value of about **0.33**: "we don't know, but a third of these knees have it".

3. Read probabilities instead of text

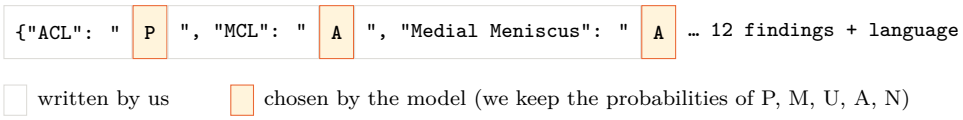


Figure 7.3: We wrote the JSON answer ourselves and let the model fill in only the letters. At each gap we kept the probabilities of all five letters (Figure 7.1). 13 tiny model calls per report instead of ∼110 generated tokens, and no parsing errors.

The soft label is the probability-weighted value:

$$\text{label} = \sum_{\text{letter}} p(\text{letter}) \cdot \text{value}(\text{letter})$$

Example Study K, ACL: $p(\text{P}) = 0.95$, $p(\text{M}) = 0.02$, $p(\text{U}) = 0.02$, $p(\text{A}) = 0.01$, $p(\text{N}) = 0.00$.
Label $= 0.95 \cdot 1 + 0.02 \cdot 0.2 + 0.02 \cdot 0.5 + 0.01 \cdot 0 =$ **0.964**.
Study K, Synovitis: $p(\text{N}) = 0.97 \Rightarrow$ label $\approx 0.97 \cdot 0.33 \approx$ **0.32**: "unknown, prior about one in three".

4. A bigger model, made affordable



Figure 7.4: 81 % of every prompt is the same instructions. Their internal state (the **KV cache**) was computed once and reused for every report, which made the 32B model affordable on two T4s.

Safety rails. Self-tests at start-up (the shortcut gives the same numbers as the plain model); the 58 gold reports first, and the 4,407-report run only if their macro AUC reached 0.84 (a **gate**); a time limit of 10.5 h with resume, because Kaggle kills sessions at 12 h and a killed run saves nothing.

7.5 Results

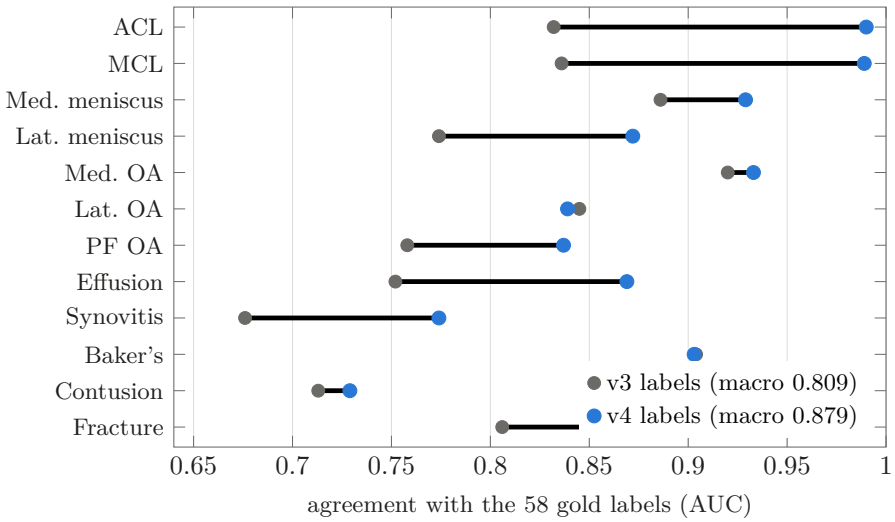


Figure 7.5: How well each label set agrees with the doctors, per finding. The threshold-aware v4 fixed ACL and MCL (almost perfect: reports state ligament tears clearly), and helped effusion, fracture and PF OA. Synovitis and contusion stayed weak.

In numbers Same image model, only the labels changed: **0.889 → 0.905** on the public leaderboard. On gold, the model's ACL went 0.887 → 0.934, MCL 0.810 → 0.887, PF OA 0.799 → 0.900. The biggest single gain of our whole project.

7.6 What the labeller actually answered

	P	M	U	A	N
ACL	11	14	1	66	8
MCL	1	14	2	74	9
Med. meniscus	35	20	2	37	6
Lat. meniscus	11	15	2	62	11
Med. OA	22	19	1	31	27
Lat. OA	12	16	1	36	36
PF OA	25	18	1	29	28
Effusion	25	35	2	29	10
Synovitis	10	3	2	1	85
Baker's	12	15	0	25	48
Contusion	15	13	4	44	24
Fracture	4	0	1	64	31

Figure 7.6: The labeller's most likely letter for each finding, in % of the 4,349 report-only studies. Synovitis is "not mentioned" (N) in 84 % of reports. MCL is P in only 1.4 % (the threshold wording was too strict: 6 of the 9 gold-positive MCL studies were answered M).

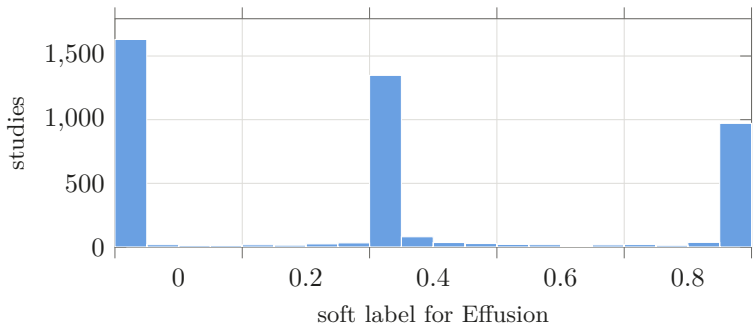


Figure 7.7: What the v4 labels look like for Effusion. Three peaks: clearly no (0, 38 %), clearly yes (1, 22 %), and a large group at **0.44**: reports that mention an effusion without saying how much (answer U). The value of U for Effusion, measured on the gold studies, is 0.44: a bit less than half of such knees had a moderate or large effusion, so 0.44 is the honest label. Soft-label training (Figure 5.7) uses these in-between values directly.

7.7 What reports cannot do

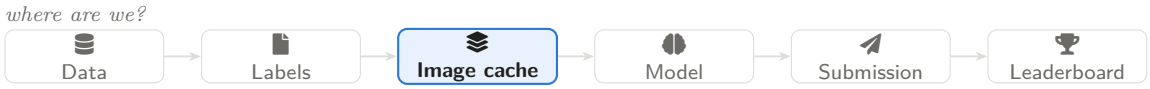
Key idea A report-based label can only be as good as the report. Where reports are explicit (ACL, MCL) our labels reached 0.99. Where they are silent or vague (Synovitis, Contusion, Lateral OA) no prompt could fix it: **the information is in the images, not in the text.** This is exactly where the winners' approach (reading the images as well) made the difference.

Checkpoint

- ☑ I can explain the five letters and how a soft label is computed from them.
- ☑ I know why reading probabilities beats parsing generated text.
- ☑ I know which findings reports label well and which they cannot.
- ? Q7.1: Why is “not mentioned” not the same as “absent”? Give a finding where the difference is large.
- ? Q7.2: v4 labels improved ACL from 0.83 to 0.99 agreement. Why did the image model’s gold ACL only go from 0.89 to 0.93?
- ? Q7.3: What does the “gate” protect against?

Chapter 8

Step 2: images into a cache



In one minute

- Reading DICOM files is slow, so we did it **once** and saved every study as three small 3D arrays: the **cache**.
- Per study: pick 3 series (sagittal, coronal, axial, fluid-sensitive), **sort** the slices, **rotate/flip** them to one convention, **normalise** brightness, **crop** 140 mm around the knee, **resize** to 24 slices of 256×256 pixels.
- The same Python file prepares the hidden test images inside the submission, so training and test images are made **identically**.

8.1 Why a cache?

Training reads every study 10 times per model, and we trained 5 models per experiment. Decoding hundreds of thousands of DICOM files each time would take hours. Decoding once (∼1.5–2 hours of CPU) and saving compact arrays makes every later experiment fast.

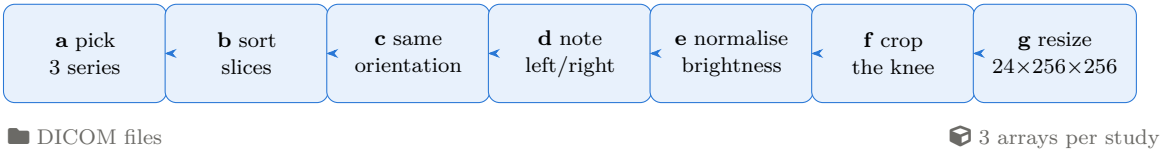


Figure 8.1: The seven steps, applied to every study. The code lives in one file, `knee_preproc.py`, saved together with the cache.

8.2 a. Pick one series per role

A study has about 5.5 series. We keep three **roles**: `sag_fs`, `cor_fs`, `ax_fs` (sagittal, coronal, axial; fluid-sensitive). `train_series.csv` says what each series is; a small score picks the best:

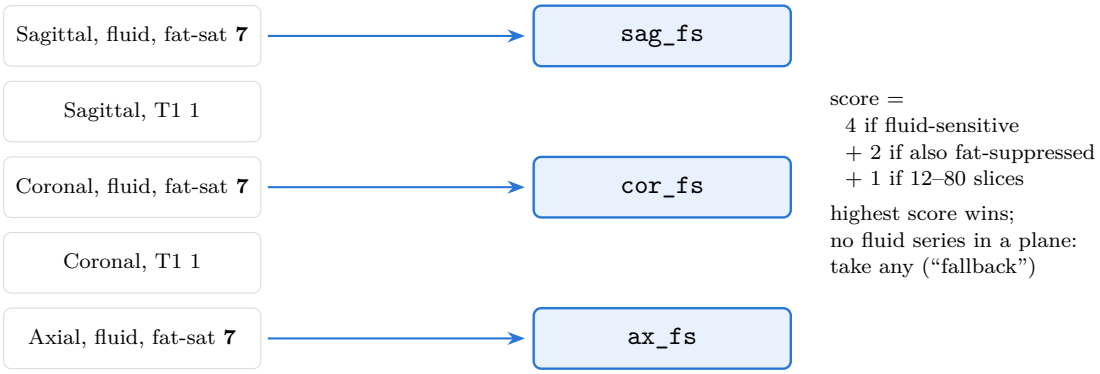


Figure 8.2: Series selection for one study. All three roles were available in 100 % of training studies.

8.3 b. Sort the slices

File names are random IDs. The only reliable order is the physical one: project each slice's `ImagePositionPatient` onto the slice direction and sort by that number.

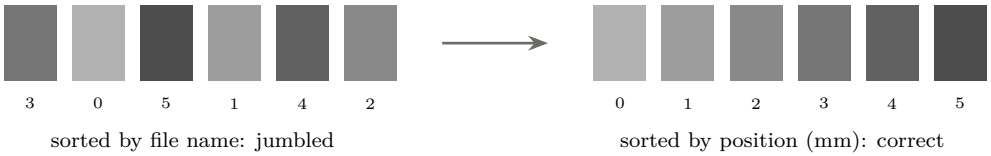


Figure 8.3: Wrong slice order turns a 3D stack into noise for anything that looks at neighbouring slices.

8.4 c. One orientation for everybody

Sites store the same plane in different ways: rotated, mirrored, upside down. The header's `ImageOrientationPatient` says how, so every series can be flipped or transposed into one convention (for sagittal: front of the knee on the left, head at the top).

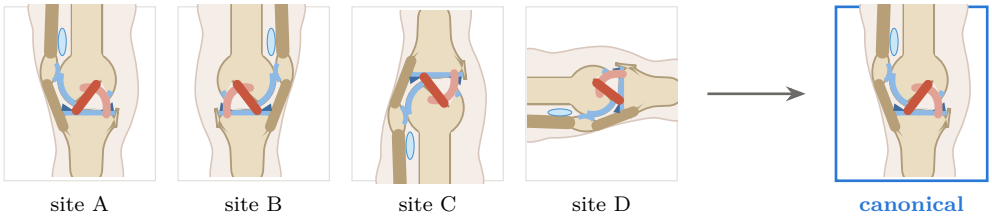


Figure 8.4: Four ways the same sagittal image can be stored, and the one convention every series is turned into. We checked that five real storage variants of one series produced identical output.

8.5 d. Left or right knee: written down, not changed

For a right knee the medial side is towards the patient's left, for a left knee the other way. The cache keeps the patient's own orientation and records the `Laterality` tag (present in 49.8 % of studies) so a later step can mirror left knees if wanted. We tried that in Chapter 11 ("v7").

8.6 e. Normalise the brightness

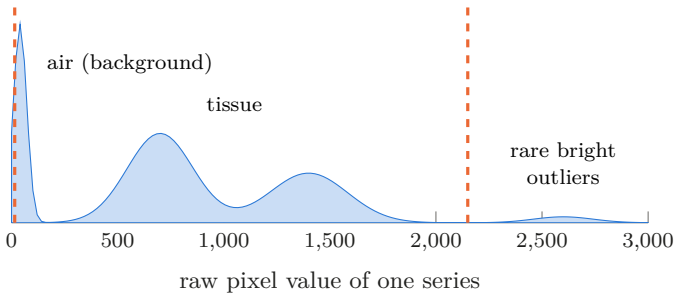
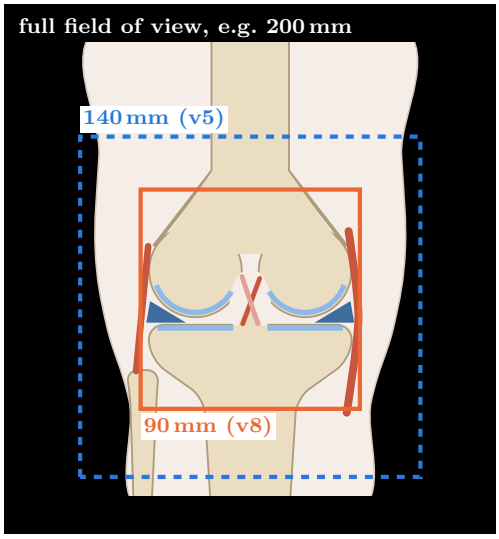


Figure 8.5: Per series: find the 0.5th and 99.5th percentile, map that range to 0–255, clip the rest. Outliers no longer decide the contrast, and every series ends up on the same scale (compare Chapter 3).

8.7 f. Crop the knee

The field of view varies between sites (often 150–200 mm), and much of it is thigh and shin. We found the knee automatically and cut a fixed square around it:



How the knee is found:

1. average all slices of the series into one image;
2. "tissue" = pixels brighter than 30 % of the 99th percentile;
3. the tissue box = rows and columns that are at least 5 % tissue;
4. centre a 140 mm square on that box (pad with black if the image is smaller).

We stored how much tissue fell outside (`tissue_outside`) to check that no knee was cut.

Figure 8.6: The crop window. The bigger the window, the less detail per millimetre after resizing to 256 pixels.

8.8 g. Resize to one shape

Every series becomes exactly **24 slices of 256 × 256 pixels**, stored as 8-bit integers (0–255): linear interpolation between slices, smooth (anti-aliased) resizing within a slice.

Cache	Window	Pixels	mm/pixel	Result (Chapter 11)
first (v1–v4)	whole field of view	224	∼0.7–0.9	0.905 public
v5	140 mm around the knee	256	0.55	**0.918** public
v6	130 mm	320	0.41	+0.002–0.004 val.: stopped
v8	90 mm on the joint	256	0.35	prepared; result not recorded
winners	140 mm; 112 mm on the joint	384–512	0.27–0.36	0.960 private

In numbers One study = 3 roles × 24 × 256 × 256 bytes ≈ 4.7 MB, so the whole training set is ∼21 GB in memory. Kaggle notebooks may output at most 500 files and 20 GB, so the cache was written as **45 compressed shards** of 100 studies each.

8.9 The same code at test time

Key idea The submission notebook imports the *same* `knee_preproc.py` from the cache dataset and runs it on the hidden test DICOMs. Any difference between training and test preparation (another resize method, another orientation rule) would silently cost score, and you would never see why.

Watch out 24 slices per series is a compromise. Native series have 20–40 slices; resampling to 24 blurs thin structures across slices. The winners kept up to 64 *native* slices and higher in-plane resolution, and used five series instead of three. That costs much more GPU time, which on Kaggle's free T4s we did not have.

Checkpoint

- ☑ I can list the seven preparation steps and say why each is needed.
- ☑ I know why the cache module is reused in the submission.
- ? Q8.1: Why normalise each series separately instead of the whole dataset at once?
- ? Q8.2: Cropping from 200 mm to 140 mm and keeping 256 pixels: how much finer does each pixel get?
- ? Q8.3: The cache does not mirror left knees. What would go wrong if it mirrored only the sagittal series?

Chapter 9

Step 3: the model



In one minute

- Our model (`KneeNet`): each of the 3 series gives **8 slice triplets**; one shared **EfficientNet-B0** turns each triplet into a feature vector; **attention pooling** makes one vector per series; a small **MLP** turns the three vectors into 12 outputs.
- Trained on **soft labels** with a masked BCE loss, **10 epochs**, about **44 minutes per fold** on one T4.
- Kaggle's GPUs taught us three hard rules: **T4 not P100**, **one GPU per process**, **fp16 not bf16**.

9.1 The architecture

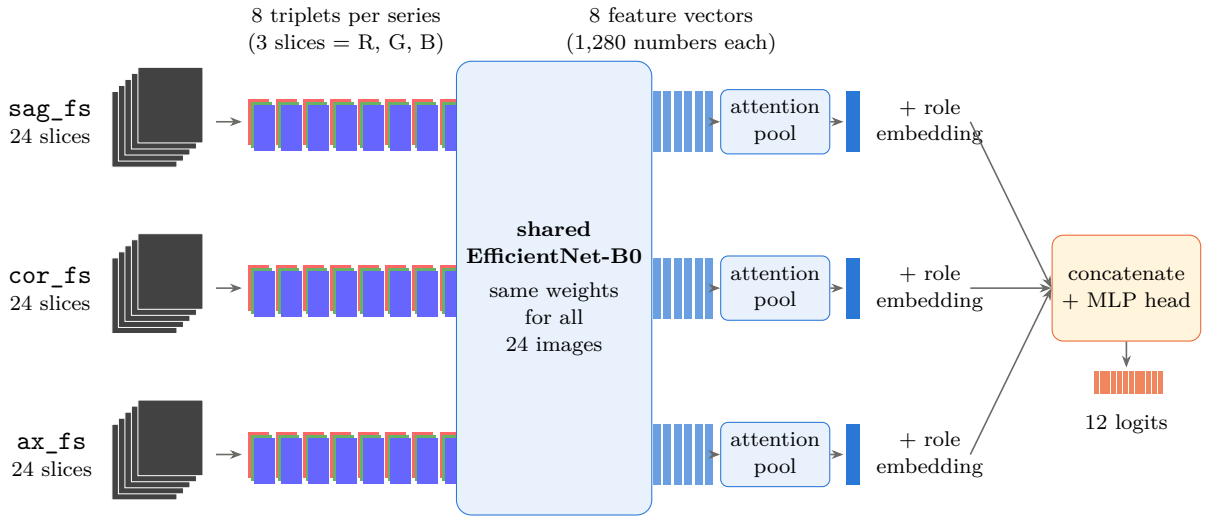


Figure 9.1: `KneeNet`. 3 series × 8 triplets = 24 images per study go through the same backbone; pooling and the head then reduce them to 12 numbers. With a batch of 8 studies, the backbone sees 192 images per step.

Slice triplets

From the 24 cached slices we pick 8 evenly spaced centres (slice 1, 4, 7, …, 22). Each centre plus its two neighbours becomes one 3-channel image, exactly the shape an ImageNet backbone expects (Figure 5.4). During training every centre moves by −1, 0 or +1 at random, a cheap

form of augmentation.

Attention pooling, role embedding, mask

- **Attention pooling** (Figure 5.5): a two-layer mini-network scores each of the 8 triplet features; the softmax of the scores gives the weights; the weighted sum is the series feature.
- **Role embedding**: a learned vector added to the series feature, so the head knows which feature came from which plane.
- **Role mask**: if a series is missing, its feature is set to zero and a 0/1 flag tells the head. (In practice all three were almost always present.)

The head

`Dropout(0.3)` → `Linear(3×1280 + 3 → 512)` → `GELU` → `Dropout(0.2)` → `Linear(512 → 12)`

Dropout switches off a random share of the numbers during training, another defence against memorising.

9.2 Loss and labels

- Targets: the v4 soft labels (Chapter 7), one value between 0 and 1 per finding.
- Loss: binary cross-entropy on soft labels, with missing labels masked out (Figure 5.7).
- The **58 gold studies are never trained on**. They are predicted by the 5 fold models together and used only as an outside check.

9.3 Augmentation, on the GPU

All augmentation runs on the GPU, per series, so the CPU never slows training down:

- small rotation (±10°), zoom (±10 %) and shift (±5 %);
- brightness, contrast and gamma changes;
- **joint knee mirror**, with probability 0.5:

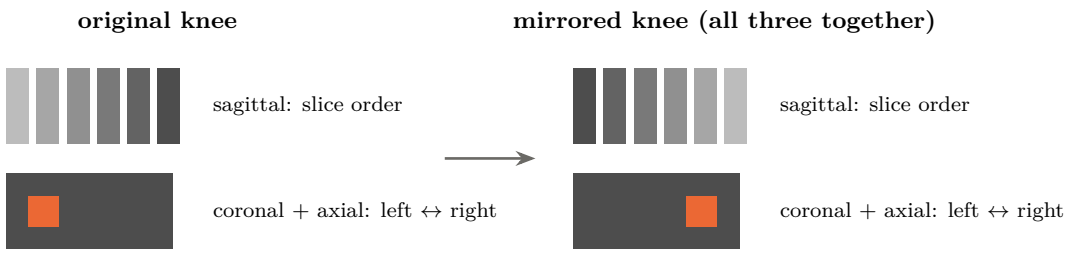


Figure 9.2: A mirrored knee looks like the other leg. Sagittal slices are reversed in order; coronal and axial images are flipped. Doing it to only one series would create an impossible knee.

Why? Mirroring makes the model see twice as many "different" knees. The price: the model can no longer tell a left knee from a right one, and therefore medial from lateral by position alone. We tested whether that hurt in Chapter 11 (it did not).

9.4 Training settings

Backbone	EfficientNet-B0, ImageNet-pretrained (later also ConvNeXt-Nano)
Optimizer	AdamW, learning rate $3 \cdot 10^{-4}$, weight decay 10^{-2}
Schedule	1 warm-up epoch, then cosine decay; 10 epochs
Batch	8 studies = 192 triplet images
Precision	fp16 mixed precision, `channels_last` memory layout
Data loading	224 px: whole cache preloaded into RAM (15.9 GB of ~32 GB); 256 px: unpacked once to local disk and memory-mapped
Validation	5-fold GroupKFold by site (31 groups); mirror TTA
Speed (one T4)	224 px: 0.41 s/step, 33 min/fold. 256 px: 0.53 s/step, 44 min/fold

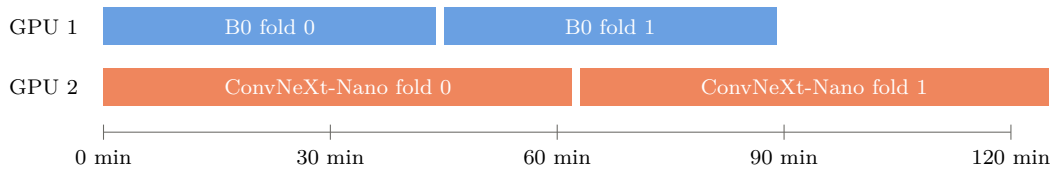


Figure 9.3: Our training notebook ran *jobs* as separate processes, one per GPU, so both T4s were busy.

9.5 Three hard rules from Kaggle's GPUs

Watch out

1. **Use the T4, not the P100.** Current PyTorch no longer ships code for the P100's older architecture.
2. **One GPU per process, no `nn.DataParallel`.** Splitting batches across two T4s made a step 18× slower (7.5 s instead of 0.4 s) and crashed the kernel. A 5-minute speed diagnostic found it after two blind fixes had failed. **Measure before fixing.**
3. **fp16, not bf16.** The T4 has no fast bf16 hardware; bf16 ran about 3× slower.

In numbers Kaggle gives about 30 GPU hours per week and kills sessions after 12 hours (a killed run saves nothing). One 5-fold training run of B0 at 256 px costs about 3.7 GPU hours. The budget, not ideas, set the pace of the project.

9.6 First results of the 256 px model

Fold 0, 256 px, 140 mm crop	validation AUC (903 studies)	gold AUC (58)	time per fold
EfficientNet-B0	0.861	0.886	44 min
ConvNeXt-Nano	0.857	0.898	62 min
rank average of the two	–	**0.903**	

Validation AUC compares with our own report labels (rounded at 0.5), gold AUC with the doctors. They disagree about which backbone is better: with this much noise (Figure 6.5) neither number alone decides.

Checkpoint

- ☑ I can draw `KneeNet` from series to 12 outputs.
- ☑ I know why mirroring must flip all three series together.
- ☑ I know the three GPU rules and why they exist.
- ? Q9.1: How many images does the backbone process for one batch of 8 studies?
- ? Q9.2: Why is the gold set predicted by all 5 fold models, while each training study gets one out-of-fold prediction?
- ? Q9.3: B0 is better on validation, ConvNeXt-Nano on gold. Which would you keep?

Chapter 10

Step 4: the submission



In one minute

- The submission notebook runs **offline** on hidden data: everything it needs is attached as datasets.
- **Safety net first**: write a valid answer file before doing anything risky, then improve it.
- Each test study is prepared by the **same** `knee_preproc.py`, predicted by every attached model group (with mirror TTA), and the groups are **rank-blended**.
- To use a public notebook, we wrote a "member cell" that blends our ranks into its predictions.

10.1 The flow

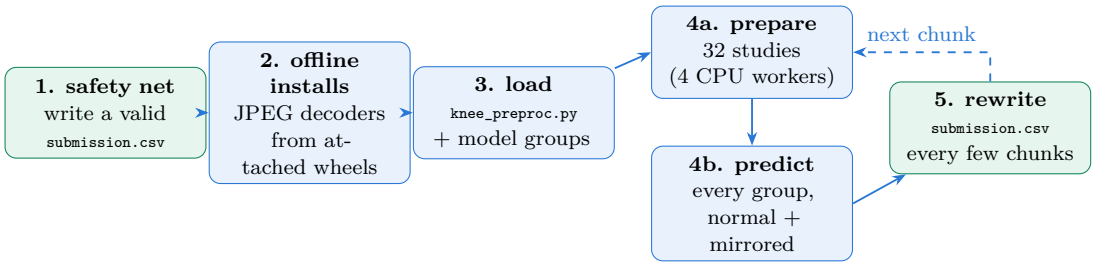


Figure 10.1: The submission notebook. Steps 4a–5 repeat until all ~1,300 test studies are done or the time limit approaches.

10.2 Safety net first

Key idea A code competition punishes crashes harshly: a crash means no score, discovered hours later. So the first thing the notebook does is write a complete, valid answer file (one guess per finding, the training prevalence). Every study is then processed inside its own `try/except`; a study that fails keeps its guess, and the run goes on.

Watch out Two of our runs of the public-notebook version failed only because a stray one-line cell (containing the text `[code]`, a paste accident) sat before our cell. A failed commit run means a failed scoring run, but you only learn that ~6 hours later. **Check the cell list before you submit.**

10.3 Offline installs

Some sites store their DICOM pixels compressed (JPEG Lossless, JPEG 2000). Decoding them needs extra Python packages, and the scoring run has no internet. We downloaded the packages' installation files (**wheels**) once into a Kaggle dataset, `knee-dicom-wheels`, and the notebook installs them from there.

10.4 Model groups and rank blending

Every attached model dataset that contains a `train_config.json` is one **group** (for example "v5 B0", "v5 ConvNeXt-Nano"). Within a group, the 5 fold models and the 2 views (normal + mirrored) are averaged. Between groups, predictions are combined **by rank**, per finding:

study	model 1	model 2	prob. avg.	rank 1	rank 2	rank avg.
A	0.95	0.52	0.735	4	3	**3.5**
B	0.10	0.55	0.325	2	4	**3.0**
C	0.40	0.50	0.450	3	2	**2.5**
D	0.05	0.49	0.270	1	1	**1.0**

probability average: order A, C, B, D = model 1's order; the cautious model 2 is ignored

rank average: order A, B, C, D; both models count equally

Figure 10.2: Why rank blending. Model 2 is cautious (all its predictions are close to 0.5), so averaging probabilities lets model 1 decide alone. Ranks put both on the same scale. Since AUC only cares about order, nothing is lost.

`BLEND_WEIGHTS` can give a group a different weight per finding (for example, weight 0 for the older v3 group on ACL, MCL and PF OA, where its labels were known to be bad).

10.5 Plugging into a public notebook: the "member cell"

By early October the public leaderboard was crowded at ~0.94 by copies (**forks**) of one public notebook that ran several strong published models (Chapter 11). We forked it and added one cell of our own:

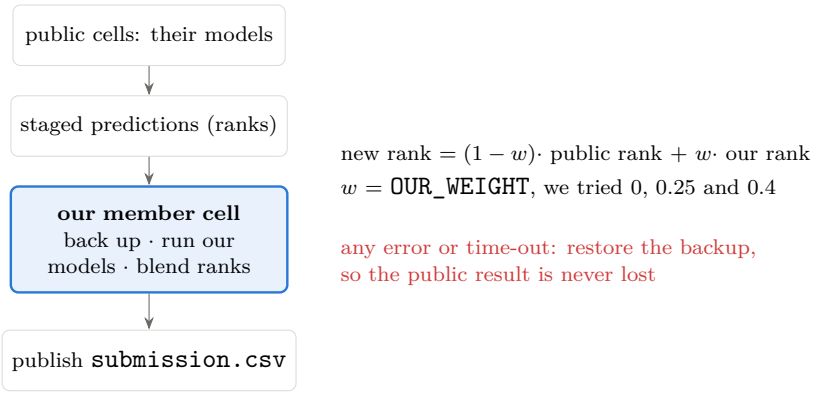


Figure 10.3: The member cell sits just before the public notebook's final "publish" cell. It only touches the staged prediction file and restores it if anything goes wrong.

In numbers

- A commit run of our own notebook (3 sample studies) took about 1–8 minutes, mostly loading models.
- Scoring a submission of the public notebook took ∼6 hours before the result (or an error) appeared.
- Our member cell had a time limit (1.5–2 hours, or 20 minutes before the public notebook's own budget).

Checkpoint

- ☑ I can explain the safety-net pattern and why every study gets its own `try/except`.
- ☑ I can rank-blend two models by hand.
- ? Q10.1: Why must the test images be prepared with exactly the training cache's code?
- ? Q10.2: Two models disagree completely on one finding. What does the rank average look like?
- ? Q10.3: What does the member cell do if our models crash halfway?

Chapter 11

Everything we tried



In one minute

- Own pipeline: **0.882** → **0.889** → **0.905** → **0.918** on the public leaderboard. The two real jumps came from **better labels** (v4) and **cropping the knee** (v5).
- A forked public notebook scored **0.943**; adding our models to it never helped.
- Higher resolution (320 px) and a canonical knee side gave **nothing measurable**; we stopped each after one fold, by rules written in advance.
- Final result: **0.927 private**, rank **2,968 of 6,425**. The winners: 0.960.

11.1 The scoreboard

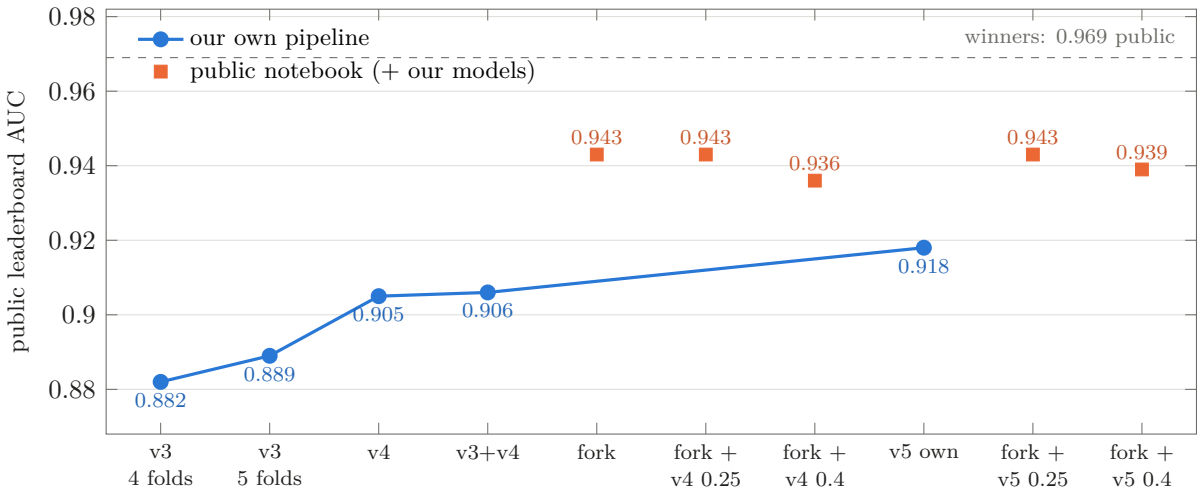


Figure 11.1: Every scored submission we recorded, in order. Public leaderboard, so differences below ∼0.005 are noise.

11.2 Experiment by experiment

E1 Baseline: v3 labels, whole knee, EfficientNet-B0

Idea. Get a complete, working pipeline end to end before improving anything.

What we did. v3 labels (Chapter 7); whole field of view resized to $24 \times 224 \times 224$; the `KneeNet` of Chapter 9; 4, then 5 folds.

Result. **0.882** (4 folds) and **0.889** (5 folds) public; gold macro 0.876.

Verdict. Works. Next question: where is it weak, and why?

E2 Detective work: compare model and labels per finding

Idea. If the model is only as good as its labels on some findings, better labels are the lever there.

What we did. Scored the labels themselves against the 58 gold studies and put them next to the model's gold AUC, per finding. Compared with numbers that other teams published for hosted-LLM labels.

Result. On ACL, MCL, PF OA and the menisci the model sat at the level of its labels, and those labels were 0.07–0.19 below what a strong labeller reached. On Effusion, Contusion, Fracture and Baker's the model was already far above its labels.

Verdict. Rebuild the labels → E3. A cheap analysis that pointed to the biggest gain.

E3 v4 labels

Idea. Follow the organisers' thresholds, keep "not mentioned" separate, read probabilities (Chapter 7).

What we did. New labeller with Qwen2.5-32B; same images, same model, retrained 5 folds.

Result. Label agreement 0.809 → 0.879. Model: **0.905** public (+0.016), gold 0.884. ACL, MCL and PF OA improved by 0.05–0.10 on gold. The menisci did not move.

Verdict. Kept. The menisci not moving suggested an image problem (too little detail), not a label problem.

E4 Blend the v3 and v4 models

Idea. Two label sets = two different opinions; average them.

What we did. Rank blend of the two 5-fold groups, v3 switched off for ACL, MCL and PF OA.

Result. Gold 0.884 → 0.896 (within noise: 90 % interval −0.005 to +0.014). Public **0.906** (+0.001).

Verdict. Tiny gain. Same architecture + same images agree too much for blending to matter.

E5 Fork the best public notebook

Idea. The public leaderboard was crowded at ∼0.94. Find out why.

What we did. Read the public write-ups: the 0.94 band consisted of forks of one inference notebook using published weights of strong models (CoAtNet-RMLP-2, 2.5D attention, 336–384 px, 44–64 slices from 5 series, plus DINOv2 and RadImageNet models for diversity). Forked it unchanged.

Result. **0.943** public, with no training of our own.

Verdict. Our base for the final submission, and a reality check: the gap was input size, slice count and model size.

E6 Our models as a member of the public ensemble

Idea. Our pipeline differs in labels and preprocessing; diversity should help an ensemble.

What we did. The member cell (Figure 10.3) with our v3+v4 blend, weights 0.25 and 0.4.

Result. 0.25: **0.943** (no change). 0.4: **0.936** (worse).

Verdict. A 0.906 model does not help a 0.943 ensemble. A member must be both *different* and *nearly as strong*. See Figure 11.2.

E7 v5: crop the knee, 256 px, add ConvNeXt-Nano

Idea. More pixels on the joint should help the menisci (E3's diagnosis).

What we did. New cache: 140 mm window around the knee, 256 px (0.55 mm per pixel instead of ∼0.8). Trained EfficientNet-B0 and ConvNeXt-Nano, 5 folds each, rank-blended.

Result. Fold 0: lateral meniscus on gold 0.784 → 0.876. Own pipeline **0.918** public (+0.012). As a member of the fork: 0.943 (weight 0.25) and 0.939 (0.4).

Verdict. Kept as our own best. Still not strong enough to lift the fork.

E8 v6: 320 px

Idea. If cropping helped, even more pixels might help more.

What we did. 130 mm window at 320 px (0.41 mm/px). Fold 0 of B0 and ConvNeXt-Nano. Stop rule written in advance: continue only if validation AUC gains ≥ +0.005.

Result. B0 +0.004, ConvNeXt-Nano +0.002, gold unchanged; cost 3.7 GPU hours for fold 0 of both models (ConvNeXt 1.8 s/step).

Verdict. Stopped. Folds 1–4 would cost ∼10 GPU hours for maybe +0.003.

E9 v7: canonical knee side

Idea. Mirror augmentation makes the model blind to left vs. right, but 4 findings are side-specific (medial/lateral meniscus and OA), and our weakest included lateral meniscus and lateral OA.

What we did. A small side classifier (trained on studies with a `Laterality` tag, 49.8 %; hold-out accuracy 0.988), then mirror every left knee so all look like right knees; no mirror augmentation, no mirror TTA. Fold 0, same rule.

Result. B0 0.8616 vs. 0.8607 (a wash); ConvNeXt-Nano 0.8497 vs. 0.8568 (worse). Gold blend 0.903 → 0.8965.

Verdict. Stopped. Lesson: mirror augmentation was *regularisation*, not just side-blindness. Note that the winners did canonicalise the side, but their model also used anatomy boxes that make the side meaningful (Chapter 13).

E10 v8: a second model on a joint-only crop

Idea. A model that sees only the joint line at near-native detail (90 mm at 256 px = 0.35 mm/px) would add *input* diversity for the menisci, OA, ACL and MCL, blended per finding with the whole-knee model.

What we did. Cache and training code built and tested locally on fake data; submission code able to prepare several crops from one decoding.

Result. Not recorded: no submission was made after early October.

Verdict. Unknown. In hindsight the winners' 112 mm joint crop gave them about +0.006 on gold: a real but small effect.

11.3 Why our models could not lift the public ensemble

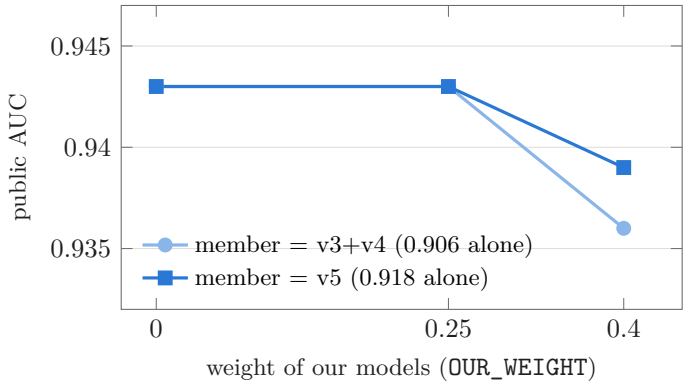


Figure 11.2: Real results. Blending our models into the 0.943 public ensemble never helped; at weight 0.4 it hurt. The stronger v5 member hurt less, which is what you expect: a member has to be close to the ensemble's strength (we estimated ∼0.92–0.93 on its own) *and* make different mistakes.

Key idea An ensemble improves when its members are **strong** and **different**. Our models were different (other labels, other preprocessing) but clearly weaker. Adding a weak member is like averaging a good forecast with a mediocre one: unless their errors are almost opposite, the average is worse than the good one.

11.4 How it ended

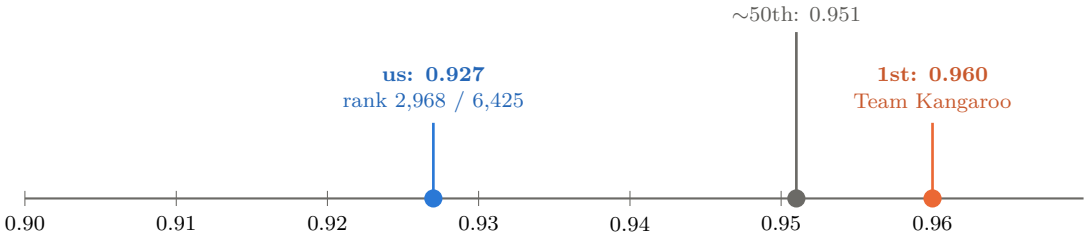


Figure 11.3: The private leaderboard (70 % of the test set). Our best selected submission and the top of the board. The top 50 teams were all within 0.01 of the winner.

In numbers

- Our public score 0.943 became **0.927** private (−0.016); the winners went 0.969 → 0.960 (−0.009). Public forks that had tuned per-finding weights on the public leaderboard may have lost more on the private part; we cannot verify which of our submissions produced the 0.927.
- Our 14 entries were all made by early October; GPU hours (∼30 per week) and scoring time (∼6 h) limited how many ideas we could test.

Watch out The big decisions that shaped this result were made early and never revisited: report-only labels (because of a rule we had not checked) and a small model on a heavily resampled input (because of the GPU budget). Every later experiment (E7–E10) tuned details within those limits.

Checkpoint

- ☑ I can name the two experiments that gave real gains and say why the others did not.
- ☑ I can explain why a weaker member did not help the public ensemble.
- ? Q11.1: What is a "stop rule" and why write it before running an experiment?
- ? Q11.2: E9 (canonical side) failed for us but the winners used it. Give one reason that could explain both.
- ? Q11.3: Our public score dropped by 0.016 on the private leaderboard, the winners' by 0.009. Name one possible reason for the difference.

Part IV

The winners

Chapter 12

The winners, part 1: labels from AI agents

In one minute

- First place: **Team Kangaroo** (Kaggle users tascj and sakaku), **0.969 public / 0.960 private**.
- They treated the competition "mainly as a labelling competition".
- For every training study an **AI agent** (a frontier model with tools) read the report **and looked at the images**, wrote down its evidence, then gave a probability per finding.
- Their labels agreed with the doctors at **0.936–0.943** AUC (ours: 0.879). The same image model got +0.013 public from the labels alone.

12.1 The big picture

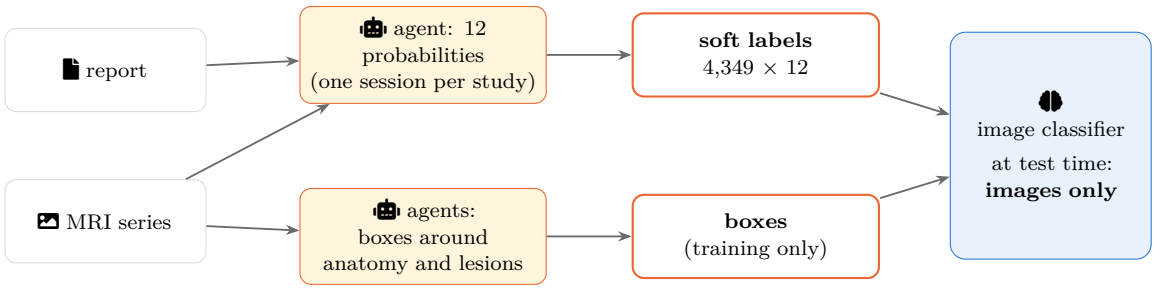


Figure 12.1: Two annotation paths feed one image model. The agents are used only to make training data; the submission runs the image model alone, offline. This chapter covers the upper path, Chapter 13 the lower.

12.2 An agent, not a prompt

Our labeller (Chapter 7) was a *prompt*: one fixed input, one answer. An **agent** is a language model running in a loop with tools: it looks at something, decides what it still needs to know, uses a tool to get it, and only answers when it is done.

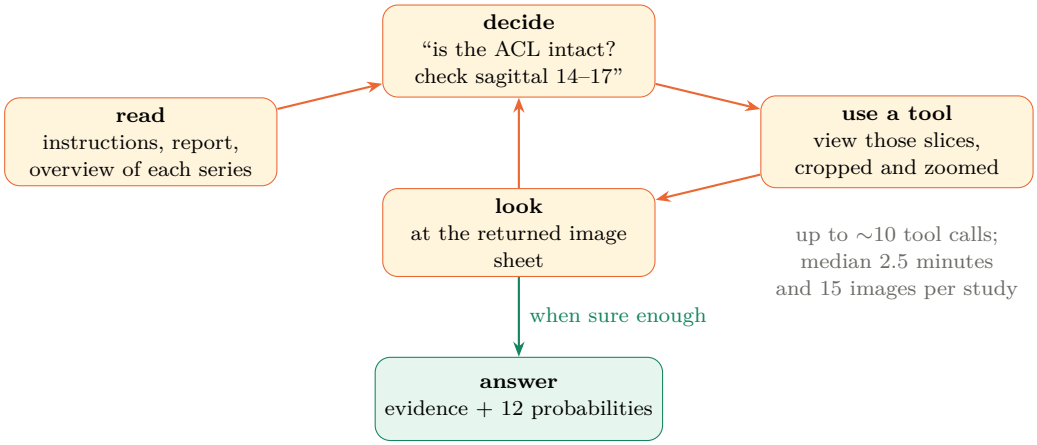


Figure 12.2: One labelling session, for one study. 4,349 such sessions labelled the training set.

Word check **Agent**: a language model that can call tools (run a script, view an image) in a loop. **Frontier model**: the strongest hosted models of the time; here OpenAI's GPT-6 Astra and GPT-6.1 Sol, driven through command-line agent tools (Codex, Claude Code and others). **Non-interactive session**: the agent runs alone, start to finish, and returns an answer in a fixed format.

12.3 The agent's workspace

Each study got its own folder, prepared before the agent started:

`<workspace>/`	
`AGENTS.md`	task, the 12 definitions and thresholds, series table, **the full report**, answer format
`overviews/`	one overview sheet per fluid series (up to 48 slices, 6 per row)
`series/`	every slice as a PNG, DICOM geometry, which series has which role
`tools/`	`view_slices`: 1–4 slices, cropped, zoomed, adjustable brightness `compare_location`: where does this point lie in another series?
`scratch/views/`	the only writable folder, for the viewer's output

Figure 12.3: The workspace (after tascj's write-up). The agent sees one study only; switching the underlying model was one line of configuration, so models and instructions could be compared fairly.

12.4 Evidence first, then a number

The required answer format made the agent write its evidence *before* its probability, per finding. In the style of their format, for our Study K:

```
"acl": {
  "report_evidence": ["Complete rupture of the anterior cruciate ligament."],
  "image_evidence": ["SAG_FLUID slices 14-17: no continuous ACL fibres between femur
                and tibia; fluid in the notch. Structural sagittal agrees."],
  "present_probability": 0.97
}
```

Why? Writing the evidence first forces the model to "look before it judges", and it makes every label checkable: a human can see which slices the agent looked at and whether the claim came from the report or from the image.

12.5 How good were the labels?

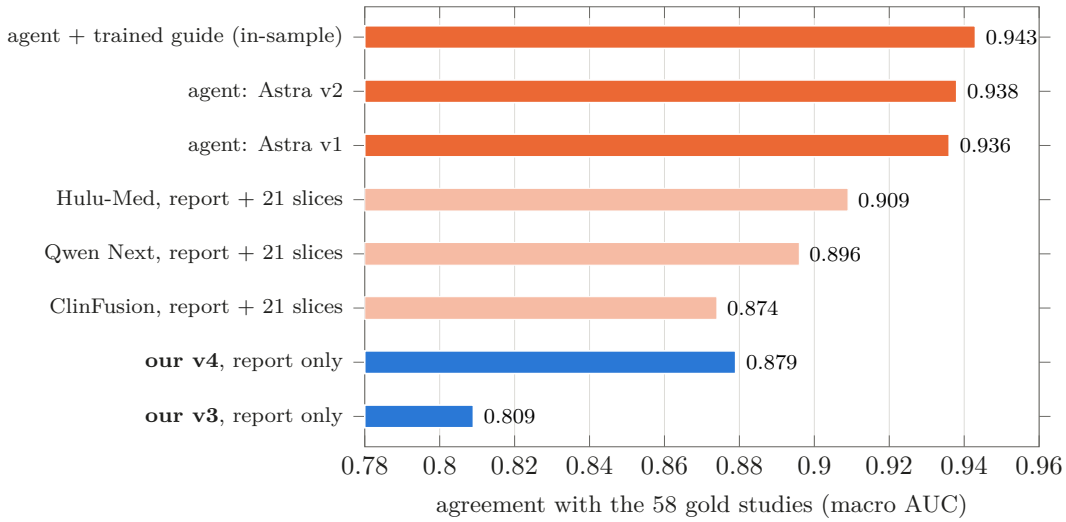


Figure 12.4: Label quality, same ruler for everybody. Single-pass readers that saw a fixed packet of 21 slices (7 per plane) already matched or beat our report-only labels; agents that chose what to look at reached 0.94. The winners' numbers are slightly optimistic: the gold studies were also used to develop their instructions.

In numbers Same image model (CoAtNet, 224 px), only the labels swapped (earlier image-aware labels → agent labels): **0.922** → **0.935** on the public leaderboard. For comparison, our own label upgrade (v3 → v4) gave +0.016.

12.6 Judging labels by more than AUC

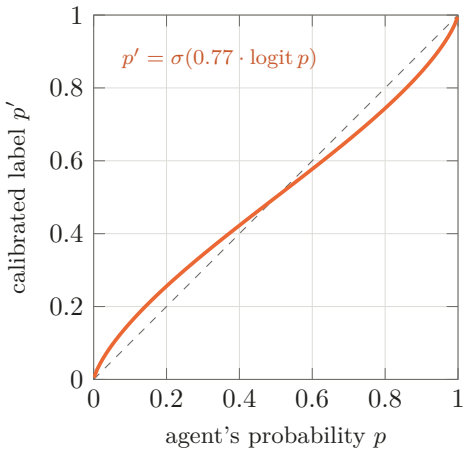
AUC only checks the order. For *training* labels the values matter too (Chapter 6), so they also counted two things:

- **Log loss**: the average BCE of the labels against the gold answers. Lower = better calibrated.
- **Confident errors**: labels that were confidently wrong (a high probability where the doctors said no, or the reverse), out of $58 \times 12 = 696$.

Model	Instructions	AUC	log loss	confident errors
GPT-6 Luna	tascj's	0.904	0.555	92
GPT-6 Luna	sakaku's	0.891	0.465	61
GPT-6.1 Sol	tascj's	0.928	0.492	63
GPT-6.1 Sol	sakaku's	0.932	0.350	44
GPT-6 Astra	sakaku's	**0.936**	**0.329**	**35**

Two lessons in one table: a stronger model helps, and **the instructions matter as much as the model**. sakaku's instructions for turning evidence into a probability halved the confident errors.

Calibration: taming over-confidence



Temperature scaling: convert the probability back to a logit, multiply by a factor k, squeeze again. With $k = 0.77$ (fitted on gold), a 0.99 becomes about 0.97 and a 0.02 about 0.05: the agents were a bit too sure of themselves. In tascj's step-by-step test this added +0.0014 on gold.

Figure 12.5: Calibrating the labels before training.

12.7 A guide that learns from the doctors

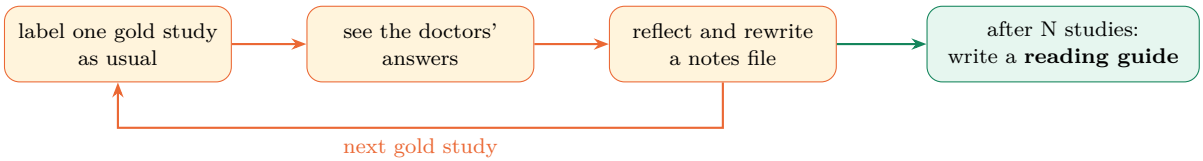


Figure 12.6: The "trained guide": one long agent session learns from the 58 gold studies and writes down what it learned. The guide is then added to the instructions for labelling everything else.

Astra labels on the 58 gold studies	AUC	log loss	confident errors
without the guide	0.936	0.329	35
+ guide learned on other folds (honest test)	0.936	0.307	**19**
+ guide learned on the same 58 (optimistic)	0.943	0.300	16

The honest test shows the real effect: same ranking, but **half the confident errors**.

12.8 Small decisions that mattered

- **Continuous, not binned.** Asking for one of six bins gave 0.883; asking for a probability 0.936. Our v4 labeller had the same instinct (probabilities over letters), but only five letters.
- **Silence is not negative, uncertainty is not positive**: the same rule as our v4 letters N and U.
- **Synovitis from the images.** Synovitis wording was removed from the report before the agent saw it, so the agent had to judge the joint lining from the images instead of copying the report.
- **Mixing label sources.** The final labels for some models averaged five sources (two Astra versions, the teammate's Astra labels, two single-pass readers) with one weight per source for all findings. Per-finding weights looked better but did not survive a leave-one-out check.

Key idea The decisive difference was not a cleverer prompt. It was **giving the labeller the images**. Reports describe what the original radiologist chose to write; the gold labels describe what is in the images. Only an image-reading labeller can close that gap, and that is exactly where our labels were weakest (Synovitis, Contusion, Lateral OA).

Watch out This was within the rules: the host had explicitly allowed hosted LLM APIs for working with the reports. The cost was real (thousands of agent sessions with a frontier model), but by the rules it counted as "reasonably accessible".

Checkpoint

- ☑ I can explain the difference between a prompt and an agent.
- ☑ I know why evidence-first answers and calibration made better training labels.
- ? Q12.1: Why might a label set with a higher AUC still be worse for training than one with a lower AUC?
- ? Q12.2: Why remove the synovitis wording from the report before the agent reads it?
- ? Q12.3: The trained guide was learned on gold studies. Why is the "other folds" row the one to trust?

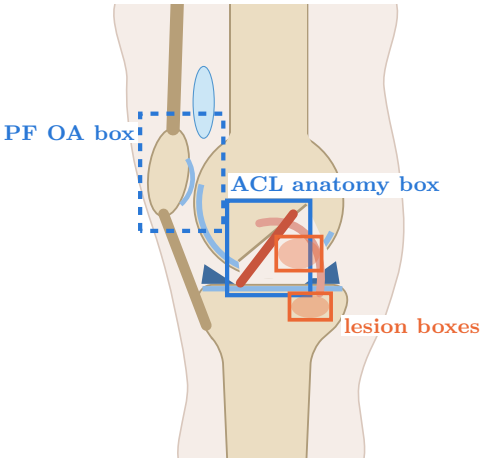
Chapter 13

The winners, part 2: models that know where to look

In one minute

- Agents also drew **boxes**: around the *anatomy* each finding concerns, and around visible *lesions*. A detector learned to draw the anatomy boxes on every series.
- **Box pooling**: each finding reads the image features only inside its own box, so "lateral meniscus" looks at the lateral meniscus.
- Training-only extras: a **lesion head** and a **report head** (predict the report's meaning from the image). Free at test time.
- Bigger input: all native slices, 384–512 px, 5 series. A validation ruler checked against the leaderboard. One model: 0.954 private in ∼4 minutes.

13.1 Two kinds of boxes



Anatomy box ("region of interest", ROI): where a finding is *assessed*, whether or not it is present. The ACL box covers the whole ligament; the PF OA box the kneecap and the femur's groove. A healthy knee has these boxes too.

Lesion box: drawn tightly around a visible problem (here a bone bruise on femur and tibia), on every slice where it is visible. Only sick knees have them.

Figure 13.1: The two kinds of spatial labels. Anatomy boxes tell the model *where to look*; lesion boxes tell it *what a problem looks like*.

13.2 From a few agent boxes to boxes everywhere

Agents could only annotate part of the data: tascj's agents drew 3D anatomy boxes for 751 studies (21 sub-sessions in parallel, managed by one main session); sakaku's drew 2D boxes slice by slice on 39,415 images from 3,823 studies; the lesion agents boxed 3,616 lesions on 1,800 studies (111,029 boxes). A **detector** then learned to predict the anatomy boxes for every slice

of every series, including the test series.

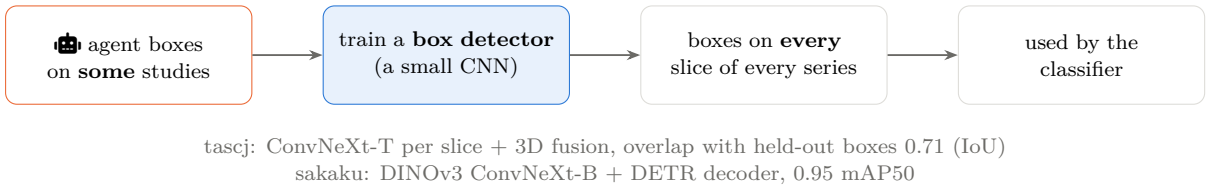


Figure 13.2: Expanding sparse annotations with a detector. tascj's detector also runs inside the submission; sakaku used boxes for training only.

Watch out **Missing is not negative.** A slice without a box can mean "structure not visible here" or "nobody annotated it". sakaku stored three states per slice and finding: *positive region*, *reliable negative* (checked: structure absent), *ignore*. Treating every empty slice as negative would teach the model the detector's misses.

13.3 Box pooling

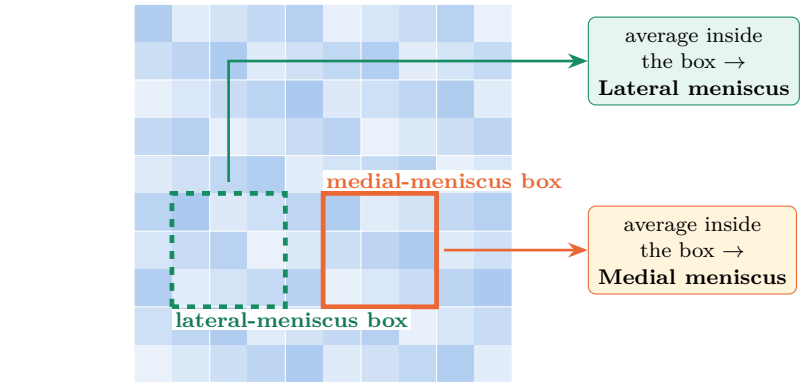


Figure 13.3: Box pooling on a coronal slice of a right knee (lateral side on the left, as in Figure 2.1). Each finding averages only the features inside its own anatomy box (weighted by how much of each cell the box covers), then one small linear layer turns that into the finding's score. A learned *in-box attention* later lets the model weight the most suspicious cells within the box more.

Why? Our model pooled whole slices (Figure 5.5): it had to discover by itself which pixels belong to the lateral meniscus. Box pooling hands it that knowledge. It also explains why the canonical side helped them and not us: with boxes, "medial" and "lateral" are explicit regions; without boxes, the side was just one more thing to learn.

13.4 Learning from boxes and reports during training only

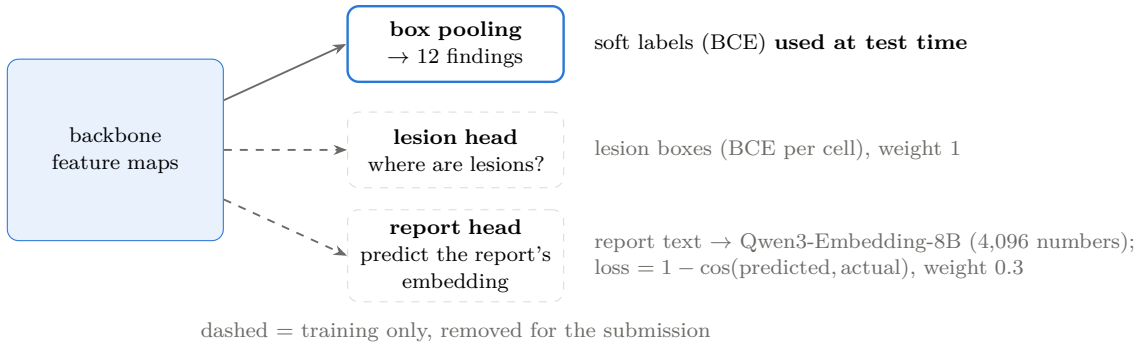


Figure 13.4: Auxiliary targets (tascj's model). They shape what the backbone learns but cost nothing at test time.

Key idea **The report-embedding head uses the report without needing it at test time.** An **embedding** is a list of numbers that captures a text's meaning. Asking the image model to predict the report's embedding forces it to learn everything the report talks about, not just the 12 findings. This needed no hosted LLM: an open-weights embedding model was enough. It is the one idea that was fully open to us under our own rule.

13.5 The input: more of everything

	us	winners
series	3 fluid-sensitive	5 roles (+ sagittal and coronal without fat suppression); one model type used every series as its own sample
slices	24, resampled	up to 64 **native** slices (or 32 sampled, 30 windows at test time)
pixels	140 mm at 256 px (0.55 mm)	140 mm at 384–512 px; joint crop 112 mm at 384 px (0.29 mm)
image unit	2.5D triplets	single slices (tascj) or triplets (sakaku)
left/right	mirror augmentation	left knees mirrored to right; side from DICOM header, else a side classifier (99.3 %)

13.6 What each piece was worth

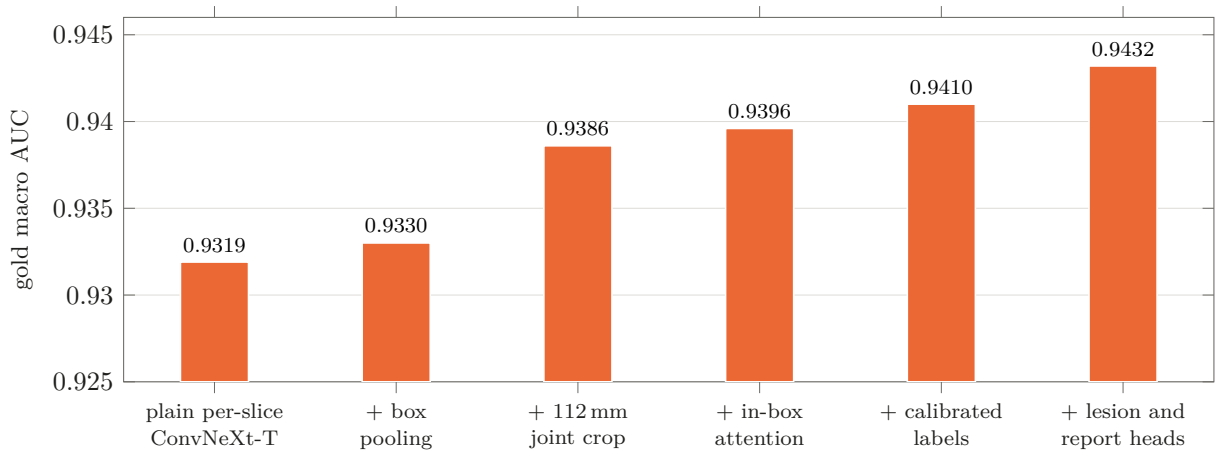


Figure 13.5: tascj's step-by-step test at 256 px, one component at a time, on the 58 gold studies (5 fold models). Note the scale: the whole architecture adds **+0.011**. The starting point, 0.932, is already far above our 0.904, because of the labels and the input.

13.7 Training

	tascj	sakaku
backbones	ConvNeXt-T, MambaOut-B+, MaxxViT-V2-B	MaxViT-V2-B, CoAtNet-rMLP-2
learning rate	$5 \cdot 10^{-5}$, constant after warm-up	backbone $8.9 \cdot 10^{-5}$ (layer-wise decay 0.9), new layers 10^{-3}, one-cycle
epochs	8, EMA 0.999	up to 30 for tests; final 19, average of epochs 15–19
loss	soft BCE + lesion + report	soft BCE + 0.05 ranking loss + box losses
precision	fp16 + `torch.compile`	bf16 + activation checkpointing

Word check **EMA** (exponential moving average): keep a slowly updated average copy of the weights and use that copy; it is usually a little better than the latest weights. **Layer-wise decay**: early backbone layers learn more slowly than late ones.

In numbers sakaku found that raising the backbone learning rate about 10× improved the public score from 0.936 to 0.944, while the 58-study gold score went *down* (0.928 → 0.922). That disagreement is what pushed the team to build a better ruler.

13.8 A ruler they could trust

Instead of the noisy 58 gold studies, they scored models on out-of-fold predictions for thousands of report-labelled studies, against their best agent labels. To decide *which* ruler to trust, they collected 20–30 leaderboard scores and kept the one that tracked the leaderboard best.

$$\text{SoftAUC} = \frac{\sum_{i \neq j} q_i\,(1 - q_j) \cdot [\,\text{pred}_i > \text{pred}_j\,]}{\sum_{i \neq j} q_i\,(1 - q_j)}$$

q = reference soft label.
A pair counts as much as "i is sick and j is healthy" is likely; ties get half credit.

Figure 13.6: SoftAUC: AUC against soft reference labels, without rounding them to 0/1 (sakaku). It keeps the labels' uncertainty in the measurement.

13.9 The final ensemble

Model (all 384 px)	Labels	Copies	Weight
tascj's half			
ConvNeXt-T (alone: 0.967 public / 0.958 private)	trained-guide Astra	5 folds	2
MambaOut-B+	calibrated Astra (+ extra heads)	full data × 3 seeds	1
MaxxViT-V2-B	same	full data × 3 seeds	1
ConvNeXt-T, 6-channel	mean of two Astra versions	full data × 3 seeds	1
MambaOut-B+, 6-channel	same	full data × 3 seeds	1
sakaku's half (alone: 0.967 public / 0.957 private)			
MaxViT-V2-B	Mix5 (five sources)	full data	1/3
MaxViT-V2-B	focus-guided Astra	full data	1/3
CoAtNet-rMLP-2	Mix5	full data	1/3
Team, 50/50			**0.969 / 0.960**

📖 **Word check** **6-channel input**: instead of 3 neighbouring slices of one series, each input image has 6 channels, one per (plane × fat suppression) kind, all resampled in 3D onto the current slice. The model sees the same spot in several sequences at once. According to tascj this added more than switching backbones did.

13.10 The efficiency notebook: 0.954 in four minutes

- One ConvNeXt-T at 256 px, trained-guide labels, all training data, one checkpoint, plus the box detector.
- Inference compiled with **TensorRT** (NVIDIA's optimiser), one process per T4, DICOM reading in C++, and only the fluid-sensitive series read (41 % less data).
- With 1,200 training studies laid out as a test set: **230 seconds** for the whole notebook. **0.964 public / 0.954 private**.

🖩 **In numbers** With the efficiency formula (Chapter 1), if the hidden-test run was as fast as their own test: $-2.16 \times 0.954 + 0.07/9 \approx -2.05$. A 1-hour notebook would need about 0.05 more AUC to match it. Kaggle's disk speed varied from ∼30 to 500+ MB/s with platform load, so they submitted when the platform was quiet.

13.11 How they worked

Both write-ups describe a workflow driven by AI coding agents: one agent was asked to develop and improve the labelling instructions against the gold studies, and tascj notes that most model ideas came from asking Claude (Opus 5.5) to propose ways to use the boxes and to run the experiments. The humans chose the direction, built the measuring stick, and decided what to trust.

Checkpoint

- ☑ I can explain the difference between an anatomy box and a lesion box.
- ☑ I can explain box pooling and why it helps side-specific findings.
- ☑ I know what auxiliary heads are and why they are free at test time.
- ? Q13.1: Why is predicting the report's embedding useful when the test set has no reports?
- ? Q13.2: The team's whole architecture added +0.011 on gold. Where did the rest of their lead over us come from?
- ? Q13.3: Why did they choose their validation ruler by correlation with the leaderboard instead of using gold?

Chapter 14

Us vs. them

In one minute

- Same overall recipe (labels from reports → image model), very different ingredients.
- Most of the gap is **labels**: theirs came from agents that looked at the images. Next: **where to look** (anatomy boxes), **input size** and **compute**.
- Several of our choices matched theirs exactly. What we lacked were the big early decisions, and one unchecked rule.

14.1 Side by side

	Us	Winners
Label source	open LLM (Qwen2.5-32B) reads the report	frontier AI agents read the report *and* view the images
Label quality (gold AUC)	0.879	0.936–0.943
Label checks	AUC on gold	AUC, log loss, confident errors; temperature calibration
Extra supervision	none	anatomy boxes, lesion boxes, report embedding
Series	3 (fluid-sensitive)	5 (+ structural)
Slices	24, resampled	up to 64 native
Pixel size	0.55 mm	0.27–0.36 mm
Left/right	mirror augmentation	canonical side + side-aware boxes
Backbones	EfficientNet-B0, ConvNeXt-Nano	ConvNeXt-T, MaxViT-V2-B, CoAtNet-2, MambaOut-B+
Pooling	attention over whole slices	box pooling inside anatomy boxes
Validation	gold-58 + report-label folds	a ruler chosen by its correlation with the leaderboard
Compute	Kaggle T4s, ∼30 h/week	bf16-capable GPUs; thousands of agent sessions
Workflow	human + AI assistant, one experiment at a time	humans + AI agents proposing and running experiments
Own model, public	0.918	0.964 (one model)
Final, private	0.927 (rank 2,968)	0.960 (rank 1)

14.2 Where the gap came from

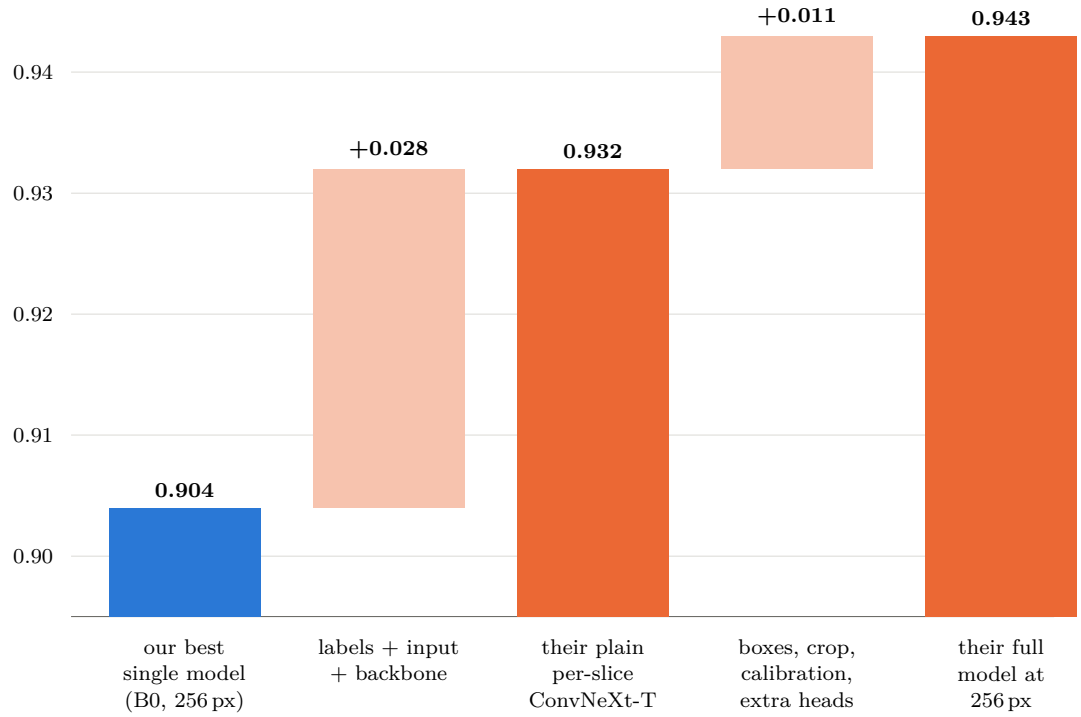


Figure 14.1: Gold macro AUC, all at 256 px with 5 fold models. The first jump (orange, light) is mostly labels; the second is everything that tells the model where to look. Caveats: different fold splits, and the winners' gold numbers are slightly optimistic. The vertical axis starts at 0.895.

On the leaderboard the gaps are larger, because the winners also scaled up: 384 px, more series, bigger backbones and an ensemble took them from a 0.964 single model to 0.969 public.

14.3 What we got right

- ✓ **Same preprocessing basics:** 140 mm field of view, per-series 0.5/99.5 percentile scaling, canonical orientation, sorting by physical position.
- ✓ **Threshold-aware soft labels**, with "not mentioned" kept separate from "absent" and "unclear" kept uncertain.
- ✓ **Probabilities, not bins**: our letter probabilities and their continuous outputs.
- ✓ **A side classifier** (98.8 % vs. their 99.3 %) and **a joint crop**: the right ideas, without the boxes that made them pay off.
- ✓ **Input diversity beats backbone diversity**: we wrote that lesson down in early October; their 6-channel models confirmed it.
- ✓ **Cheap, rule-based experiments**: one fold, a stop rule written in advance, stop when it fails.

14.4 Seven lessons

Seven lessons

1. **Read the rules threads on day one, and verify every self-imposed rule.** Our "no hosted LLM" rule was a guess. It shaped everything that followed.
2. **When the labels are weak, improve the labels, not the model.** Every model-side change we tried was worth $\leq +0.003$; better labels were worth $+0.016$ to us and $+0.013$ to them on the same model.
3. **Labels must see what the target sees.** The gold labels came from images; report-only labels cannot know what the report leaves out.
4. **Judge training labels by calibration as well as AUC.** Confidently wrong labels teach confident mistakes.
5. **Tell the model where to look.** Anatomy boxes turned "medial vs. lateral" from something to learn into something given.
6. **Build a ruler you can trust before you optimise.** 58 studies cannot resolve 0.01; the winners chose their validation by how well it predicted the leaderboard.
7. **Look at the public work early**, and at what the top teams do differently, not just at what scores 0.94.

14.5 If we started again

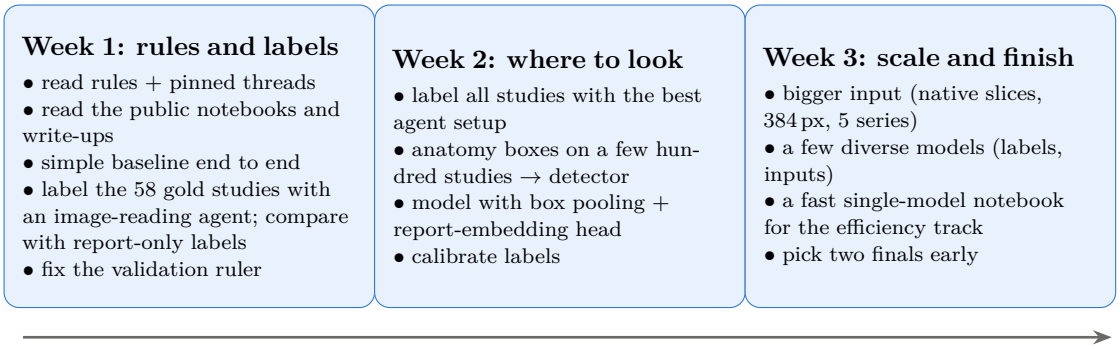


Figure 14.2: A plan built from both write-ups and our own mistakes. Most of it is about *data*, not models.

Key idea Our pipeline was a sound, well-tested version of the obvious approach, and it scored like one. The winners changed the problem itself: they made better training data, and made it in a form (probabilities plus boxes) that the model could use. In a competition with only 58 labelled examples, **that is where the points were**.

Checkpoint

- ☑ I can explain, in two sentences, why the winners scored 0.960 and we scored 0.927.
- ☑ I can name three things we did the same way as the winners.
- ? Q14.1: Which single lesson would have changed our result most, and why?
- ? Q14.2: Could we have used anatomy boxes without hosted LLMs? How?

Appendix A

Glossary

2.5D: a 2D image model fed with a few neighbouring slices as its colour channels (Fig. 5.4).

Agent: a language model that calls tools in a loop until it is ready to answer (Fig. 12.2).

Anatomy box (ROI): a box around the structure a finding concerns, present in healthy knees too (Fig. 13.1).

Attention pooling: a weighted average of several feature vectors, with learned weights (Fig. 5.5).

AUC (ROC-AUC): the chance that a random positive study scores higher than a random negative one (Ch. 6).

Augmentation: random changes to training images (rotation, brightness, mirroring) against overfitting.

Axial: slices across the leg, from bottom to top.

Backbone: the large, pretrained part of an image model that turns an image into features (Fig. 5.3).

Batch: the group of studies processed in one training step (ours: 8).

BCE (binary cross-entropy): the standard loss for yes/no outputs; works with soft labels (Fig. 5.7).

bf16 / fp16: 16-bit number formats for faster GPU maths. On a T4, use fp16.

Box pooling: averaging image features only inside a finding's anatomy box (Fig. 13.3).

Cache: our pre-processed images: 3 arrays of $24 \times 256 \times 256$ per study (Ch. 8).

Calibration: whether a predicted 0.8 really means "80 % of such cases are positive".

Code competition: you submit a notebook; Kaggle runs it offline on hidden data.

Compartment: one of three areas where cartilage surfaces meet: medial, lateral, patellofemoral (Fig. 2.2).

Confident error: a label that is confidently wrong (high when the truth is negative, or the reverse).

Coronal: slices from front to back; each slice is a front view.

Cross-validation (k-fold): train k models, each validated on a different held-out part (Fig. 5.12).

DICOM: the file format of medical images: header + pixels (Ch. 4).

Dropout: switching off random parts of a network during training, against memorising.

EMA: a slowly updated average of the model weights, often better than the latest weights.

Embedding: a list of numbers that captures the meaning of a text or image.

Ensemble: averaging several models' predictions.

Epoch: one pass over all training studies.

Fat suppression (fat-sat): an MRI technique that switches off the fat signal so that fluid and swelling stand out.

Feature map: the image-like output of a CNN layer; bright where a learned pattern occurs.

Fine-tuning: continuing to train a pretrained model on your own task.

Fluid-sensitive: a sequence in which water is bright (PD or T2, usually fat-suppressed).

Fold: one of the k parts in cross-validation.

Fork: a copy of someone else's public notebook.

Frontier model: the strongest hosted AI models of the time.

Gold labels: the trusted reference labels: here the 58 studies labelled by radiologists from the images.

GroupKFold: cross-validation that keeps related studies (here: one site) in the same fold.

Head: the small final part of a model that turns features into outputs.

ImageNet: 1.2 million everyday photos; the usual pre-training data for image backbones.

Inference: using a trained model to make predictions.

KV cache: a language model's stored internal state for text it has already read; lets a shared prompt prefix be reused.

Lateral / medial: the outer / inner side of the knee.

Learning rate: how big a step the optimizer takes per training step.

Lesion box: a tight box around a visible problem on one slice (Fig. 13.1).

LLM: large language model.

Log loss: the average BCE of predictions (or labels) against the truth; measures calibration as well as order.

Logit: the raw output of a model before the sigmoid.

Loss: one number that says how wrong the model currently is.

Macro average: the plain average of the 12 per-finding AUCs.

Masked loss: a loss that skips missing labels.

Member cell: our code cell that blended our predictions into a public notebook (Fig. 10.3).

Mirror TTA: predicting on the original and the left–right mirrored knee, then averaging.

Multi-label: several yes/no questions about the same input at once.

Multiple-instance learning: learning from a label that belongs to a group (a study) rather than to each item (slice).

OOF (out-of-fold): predictions for each training study from the model that did not train on it.

Overfitting: memorising the training data instead of learning what generalises.

PD, T1, T2: names of MRI sequences (Ch. 3).

Pretrained: already trained on other data before your task.

Public / private leaderboard: scores on ∼30 % / ∼70 % of the hidden test set; the private one decides.

Quantization: storing weights with fewer bits (e.g. 4) to save memory.

Rank averaging: blending models by averaging the ranks of their predictions (Fig. 10.2).

ROC curve: true-positive rate against false-positive rate over all thresholds.

Role: one of our three series slots: `sag_fs`, `cor_fs`, `ax_fs`.

Sagittal: slices from left to right; each slice is a side view.

Sequence: the scanner settings that decide which tissues are bright.

Series: one 3D stack of slices with one direction and one sequence.

Sigmoid: the function that squeezes a logit into a probability between 0 and 1.

Slice: one 2D MRI image; one DICOM file.

Soft label: a label between 0 and 1 that expresses uncertainty.

SoftAUC: AUC computed against soft reference labels, weighting each pair by how likely it is to be a positive/negative pair.

Stop rule: a threshold for continuing an experiment, fixed before seeing the result.

Study: one MRI exam of one knee; one row of labels.

T4: the NVIDIA GPU that Kaggle offers (two of them, 16 GB each).

Temperature scaling: multiplying logits by a constant to make probabilities less (or more) confident.

Token: a word piece; the unit a language model reads and writes.

Transfer learning: reusing a model trained on one task as the start for another.

Voxel: a 3D pixel; in knee MRI typically $0.4 \times 0.4 \times 3.5$ mm.

Weak supervision: training on cheaper, noisier labels (here: from reports).

Weights: the adjustable numbers inside a model.

Appendix B

Answers to the checkpoints

Chapter 1

Q1.1 You never see the test data, and a crash anywhere means no score at all, discovered only hours later. In a normal competition you upload a file you can check first.

Q1.2 Not necessarily. With ∼390 public studies the score wobbles by about ±0.005, so 0.003 is within noise.

Q1.3 No. +0.01 AUC is worth about 0.022 efficiency points; one hour costs about 0.11.

Chapter 2

Q2.1 No. “Small” is below the threshold; only moderate or large effusions count.

Q2.2 Medial OA, Lateral OA and PF OA: the same cartilage-loss rule (about 1 cm or more with over half the thickness lost), each in a different compartment.

Q2.3 Medial and lateral swap sides between a left and a right knee. A side-blind model cannot use position to tell the two menisci apart and has to rely on shape and context.

Chapter 3

Q3.1 On T1, edema is greyish against bright fatty marrow: hard to see. On fat-suppressed PD the marrow is dark and the edema bright: easy.

Q3.2 MRI brightness has no units; it depends on scanner, coil, sequence and patient. Each series must be normalised on its own.

Q3.3 For example the MCL, which runs along the side of the knee and is best seen on coronal slices. (PF OA and Baker’s cyst, best on axial, are other good answers.)

Chapter 4

Q4.1 File names are random IDs. Only the physical position gives the true order.

Q4.2 Not necessarily. The gold set was chosen to contain many positives, so its prevalence is higher by design. Compare labels with gold by AUC, not by prevalence.

Q4.3 The test studies have no reports. Only images are available at test time.

Chapter 5

Q5.1 Three different neighbouring slices give the network some depth context (a tear that continues across slices) at no extra cost; a copied slice gives it nothing new.

Q5.2 $p = 0.5$. The model learns “don’t be confident about this one”, which is the honest message of an unknown label.

Q5.3 Only a little. Same images and labels lead to strongly correlated mistakes; ensembles help when the members differ.

Chapter 6

Q6.1 1.0. Every sick study scores above every healthy one; only the order matters.

Q6.2 Nothing. The order of the predictions does not change.

Q6.3 Synovitis: $+0.04/12 \approx +0.0033$ on the macro AUC, against $+0.01/12 \approx +0.0008$ for Baker's.

Chapter 7

Q7.1 "Absent" means the report says the structure is normal; "not mentioned" is silence. For Synovitis the difference is large: 15 of 41 silent gold studies were positive.

Q7.2 Near-perfect labels do not make the images easier. The image model still has to see the tear, with limited resolution and capacity; and 58 gold studies measure it with noise.

Q7.3 Spending many GPU hours labelling everything with a labeller that turns out worse. The gold studies are labelled first, and the long run starts only if they are clearly better.

Chapter 8

Q8.1 Brightness differs between series, even within one exam; normalising per series removes those differences, a dataset-wide rule would keep them.

Q8.2 $200/140 \approx 1.43$ times finer (for example 0.78 mm $\rightarrow$ 0.55 mm per pixel).

Q8.3 The study would describe an impossible knee: the sagittal series says "left knee", the coronal and axial say "right knee". The model would see contradictory geometry.

Chapter 9

Q9.1 8 studies $\times$ 3 series $\times$ 8 triplets = 192 images.

Q9.2 Gold studies are never trained on, so all 5 fold models are fair predictors and their average is a free ensemble. Each training study was held out by exactly one fold model; only that model is unbiased for it.

Q9.3 Neither number alone is decisive at this noise level. Keep both and blend them, which is what we did (gold 0.903).

Chapter 10

Q10.1 Any difference (resizing, orientation) changes what the model sees; the model was never trained on such images, and the score drops without any error message.

Q10.2 The ranks cancel out: every study ends near the middle and the finding becomes close to random (AUC near 0.5 if the two models are exactly opposite).

Q10.3 It restores the backup of the public notebook's staged predictions, so the public result is submitted unchanged.

Chapter 11

Q11.1 A threshold for continuing an experiment, fixed in advance, so that noise is not talked into "progress" after the fact.

Q11.2 In their model the side is used explicitly (side-aware boxes and box pooling), so a consistent orientation helps. Our model had no use for the side, and lost the regularising effect of mirror augmentation.

Q11.3 For example: the public notebook's per-finding weights had been tuned on the public leaderboard and did not carry over to the private part; or simply noise. The winners picked models with a ruler, not by probing the leaderboard.

Chapter 12

Q12.1 AUC ignores calibration. A label set with many confidently wrong labels can rank well but teaches the model to be confidently wrong; log loss and confident errors catch that.

Q12.2 Reports rarely describe synovitis, and when they do the agent would just copy the wording. Without it, the agent has to judge from the images, like the gold radiologists.

Q12.3 A guide learned on other folds is applied to studies it has never seen: an honest test. The in-sample guide has seen the answers.

Chapter 13

Q13.1 To predict the embedding, the image features must encode what the report talks about. At test time the head is simply removed, so no report is needed.

Q13.2 Mostly from the labels, plus input and backbone: their plain per-slice model already reached 0.932 on gold, against our 0.904; then bigger inputs and ensembles on the leaderboard.

Q13.3 58 gold studies are too few (± 0.02) and sometimes disagreed with the leaderboard. A ruler that tracks 20–30 leaderboard scores is known to measure what matters.

Chapter 14

Q14.1 Checking the rules thread on day one: it would have allowed image-reading labels from the start, the largest single source of the gap.

Q14.2 Yes, more slowly: an open-weights vision-language model running on Kaggle, or a few hundred studies annotated by hand or by simple anatomy heuristics, followed by a detector trained on them.

Appendix C

Image credits and sources

MRI images (Wikimedia Commons)

The real MRI images are from Wikimedia Commons and are used under their Creative Commons licences. Arrows and labels were added to the two sagittal images in Figure 3.4; otherwise they are only scaled.

Used in	File on Wikimedia Commons	Author	Licence
Fig. 3.3, 3.4	Knee MRI PD TSE FS Sagittal.jpg	Ptrump16	CC BY-SA 4.0
Fig. 3.4	Knee MRI T1 TSE Sagittal.jpg	Ptrump16	CC BY-SA 4.0
Fig. 3.3	Knee Cor 140234 t2me2d.png	Nevit Dilmen	CC BY-SA 3.0
Fig. 3.3	Knee MRI 135826 pdfs.png	Nevit Dilmen	CC BY-SA 3.0
Fig. 2.4	MRT VKB-Riss PDW.jpg	Hellerhoff	CC BY-SA 3.0
Fig. 2.4	MRI meniscus tear.jpg	Ciernik M	CC BY-SA 4.0
Fig. 2.4	Bakerzyste MRT ax.jpg	Hellerhoff	CC BY-SA 3.0

Each file's page: `https://commons.wikimedia.org/wiki/File:` followed by the file name (spaces as underscores). Licences: `https://creativecommons.org/licenses/by-sa/4.0/` and `https://creativecommons.org/licenses/by-sa/3.0/`.

Drawings and charts

All knee drawings, diagrams and charts were made for this script. Charts of our own results use only numbers (labels, probabilities, predictions) from the project's `data/` folder, produced by `docs/script/make_figure_data.py`; no report text was used. Study K and its report are invented.

Sources

- tascj and sakaku, *1st Place Solution*, Kaggle competition write-up, 25 Oct 2026. `https://www.kaggle.com/competitions/rsna-knee-abnormality-detection/writeups/1st-place-solution`
- sakaku, *1st place solution: Better labels, anatomy-aware MRI models*, Kaggle discussion 752474, Oct 2026.
- Competition host, *Use of Commercially Hosted LLMs*, pinned Kaggle discussion 733965.
- Kaggle leaderboard of *RSNA Knee Abnormality Detection*, read on 25 Oct 2026 (private leaderboard, preliminary).
- Our project: notebooks, analysis and notes in the GitHub repository `david-pilny/RSNA-Knee-Abnormality-Detection` (`CLAUDE.md` holds every number quoted here for our pipeline).
- The public notebook we forked: `mattiaangeli/bend-the-knee-to-speedy-raptors-the-original` on Kaggle.